EASTERN INTERNATIONAL UNIVERSITY

SCHOOL OF COMPUTING AND INFORMATION TECHNOLOGY

DEPARTMENT OF SOFTWARE ENGINEERING

TRƯỜNG ĐẠI HỌC QUỐC TẾ MIỀN ĐÔNG

EASTERN INTERNATIONAL UNIVERSITY

PROJECT MOBILE

HeartHealth

Student(s)

Nguyễn Hoàng Giang - 2131200071

Supervisor(s)

Trần Văn Tài

Nguyễn Hoàng Lý

Ho Chi Minh, August 2026

ABSTRACT

In recent years, the global prevalence of cardiovascular diseases has made personal health monitoring more critical than ever. Many individuals, especially those with chronic conditions such as hypertension, diabetes, and high cholesterol, struggle to maintain a consistent routine of tracking their vital signs and following medical schedules. Traditional paper-based health diaries are inefficient, easily lost, and unable to provide meaningful trend analysis. In response to this growing need, we developed HeartHealth — a cross-platform mobile application designed to empower users in daily cardiovascular health management. Built with React Native for the mobile frontend and Spring Boot with MySQL for the backend, HeartHealth provides a comprehensive suite of features including: daily vital-sign recording (blood pressure, heart rate, SpO2, cholesterol, blood sugar, and weight), an intelligent triage screening system with risk-level assessment, interactive health statistics charts with 7-day and 30-day trend views, medication schedule reminders, doctor appointment management, and bilingual support (Vietnamese/English). The system follows a RESTful client-server architecture with a clear separation of concerns: all critical business logic (health risk assessment, data processing, OTP authentication) is handled server-side, while the mobile client focuses on delivering a premium, user-friendly experience with smooth animations and intuitive navigation.

ACKNOWLEDGEMENT

We would like to express our sincere and heartfelt gratitude to our supervisors, Mr. Trần Văn Tài and Mr. Nguyễn Hoàng Lý, for their dedicated guidance and valuable support throughout the development of this project. Their expertise and commitment have provided us with essential advice and professional insights that helped us overcome various challenges — from database design and backend API implementation to mobile UI/UX development — during the completion of this product. We also extend our appreciation to the Eastern International University and the Department of Software Engineering for providing us with the academic foundation and resources necessary to pursue this project. Once again, we are deeply thankful for all the significant contributions that made this project possible.

TABLE OF CONTENTS

LIST OF FIGURES

CHAPTER 1. Introduction

1.1. Motivations

Cardiovascular disease is the leading cause of death globally (~17.9 million deaths/year, WHO). Early detection through consistent vital sign monitoring can significantly reduce cardiac risks. However, existing health apps are either fitness-focused, lack Vietnamese localization, or don't combine metrics tracking with medical triage assessment. HeartHealth addresses this gap — a mobile-first platform specifically designed for cardiovascular patients that combines daily monitoring, risk assessment, trend visualization, and scheduling in one bilingual app.

1.2. Objectives

The primary objectives of the HeartHealth project are:

1. Build a cross-platform mobile app (React Native) for daily health metric recording with intuitive slider-based inputs.
2. Develop a RESTful backend (Spring Boot + MySQL) to securely store and process health data.
3. Implement a triage screening system classifying risk as Normal / Moderate / High / Emergency based on clinical thresholds.
4. Provide interactive line charts for 7-day and 30-day health trends.
5. Enable medication reminders and doctor appointment scheduling. • Support bilingual localization (Vietnamese / English).
5. Implement secure email-based OTP authentication via Gmail SMTP.

1.3. Overview

HeartHealth consists of:

• Mobile Frontend: React Native app with 20 screens covering authentication, screening, metric entry, statistics, scheduling, and profile management.

• Backend Service: Spring Boot REST API with 16+ endpoints across 4 controllers (Auth, Health, Goal/Schedule, Admin).

• Database: MySQL with 7 tables managed via Hibernate ORM and 12+ versioned DDL migration scripts.

Architecture: Client-server with HTTP/JSON communication. All critical business logic (triage calculations, OTP generation, data aggregation) runs server-side.

CHAPTER 2. Technical

2.1. Technology

2.1.1. Java & Spring Boot (Backend)

Java 17 with Spring Boot 4.1.0 provides the backend foundation:

- Spring Data JPA / Hibernate ORM for database access
- Spring Boot Starter Mail for Gmail SMTP OTP delivery
- Lombok (@Data, @Builder) to reduce boilerplate
- Maven Wrapper (mvnw) for build automation
- Global Exception Handler for consistent error responses

The layered architecture (Controller → Service → Repository → Entity) enforces clean separation of concerns.

2.1.2. React Native & JavaScript (Frontend)

React Native enables cross-platform mobile development from a single JavaScript codebase:

- React-navigation (stack + bottom tabs) for screen navigation
- React-native-chart-kit for line chart visualization
- React-native-vector-icons (MaterialIcons, MaterialCommunityIcons)
- React-native-community/slider for metric input controls
- React-native-async-storage for persistent local storage
- Axios for HTTP communication with the backend
- Custom LanguageContext for real-time VN/EN switching

2.1.3. MySQL

MySQL 8.0+ serves as the relational database:

- Reliable data integrity via foreign keys and NOT NULL constraints
- Seamless integration with Spring Data JPA / Hibernate
- Scalable for growing daily health metrics data
- Parameterized queries via JPA prevent SQL injection

2.1.4. Gmail SMTP & OTP Authentication

Secure authentication without third-party OAuth:

- Server generates random 6-digit OTP, stored with 5-min expiry
- OTP sent via Gmail SMTP (smtp.gmail.com:587, TLS encrypted)
- Client submits OTP for server-side validation
- Supports Registration, Login, and Password Reset flows

2.2. Strengths of the Website Based on Its Technology Stack

1. **Robust Backend**

   Spring Boot's layered architecture with Hibernate ORM ensures maintainable, testable code.

2. **Premium Mobile UX**

   React Native delivers native-quality animations, intuitive sliders, and a Pink Rose medical theme.

3. **Smart Data Layer**

   "Carry-forward" algorithm fills chart gaps; "smart merge" prevents data loss on partial daily updates.

4. **Secure Auth Pipeline**

   Server-side OTP generation, email delivery, and validation - no third-party auth dependencies.

5. **Bilingual Support**

   Real-time VN/EN switching via Context API across all 20 screens without app restart.

6. **Cross-Platform**

   Single React Native codebase for both Android deployment.

CHAPTER 3. System Design

3.1. System Workflow

3.1.1. General Use Case Diagram

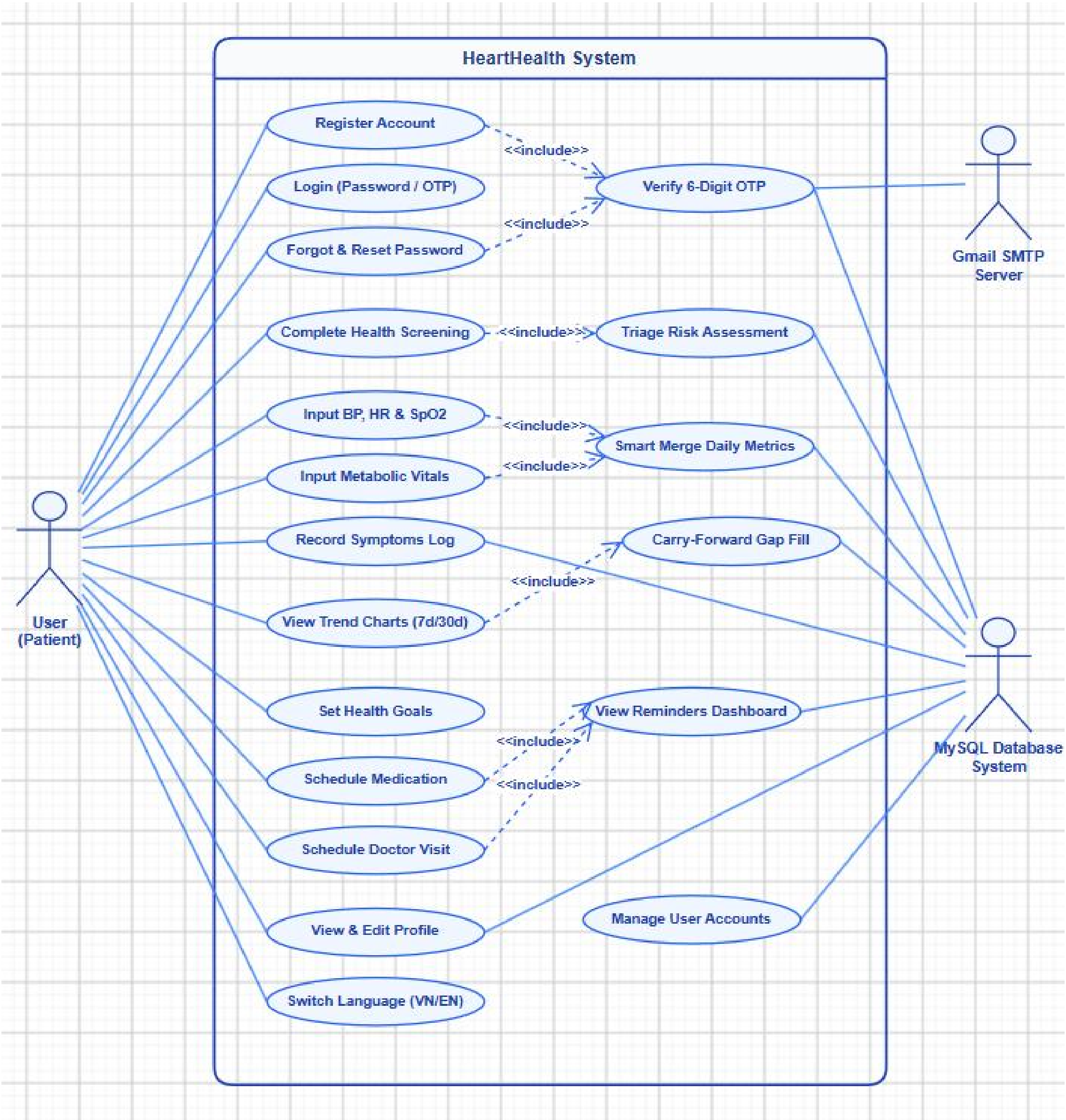


Figure 1. General Use Case

- **Actors**: User.
- **User Action**:

  Register, Login, Forgot Password, Complete Health Screening (Triage Assessment), View Dashboard, Input Daily Metrics, Record Symptoms, View Statistics Charts, Set Health Goals, Schedule Medication, Schedule Doctor Appointments, View/Edit Profile, Switch Language, Logout.
- **Database:**

  Handles persistence for all user, health, and scheduling data.

3.1.2. Authentication Use Case

❖ Authentication (Register / Login / Forgot Password / Logout)

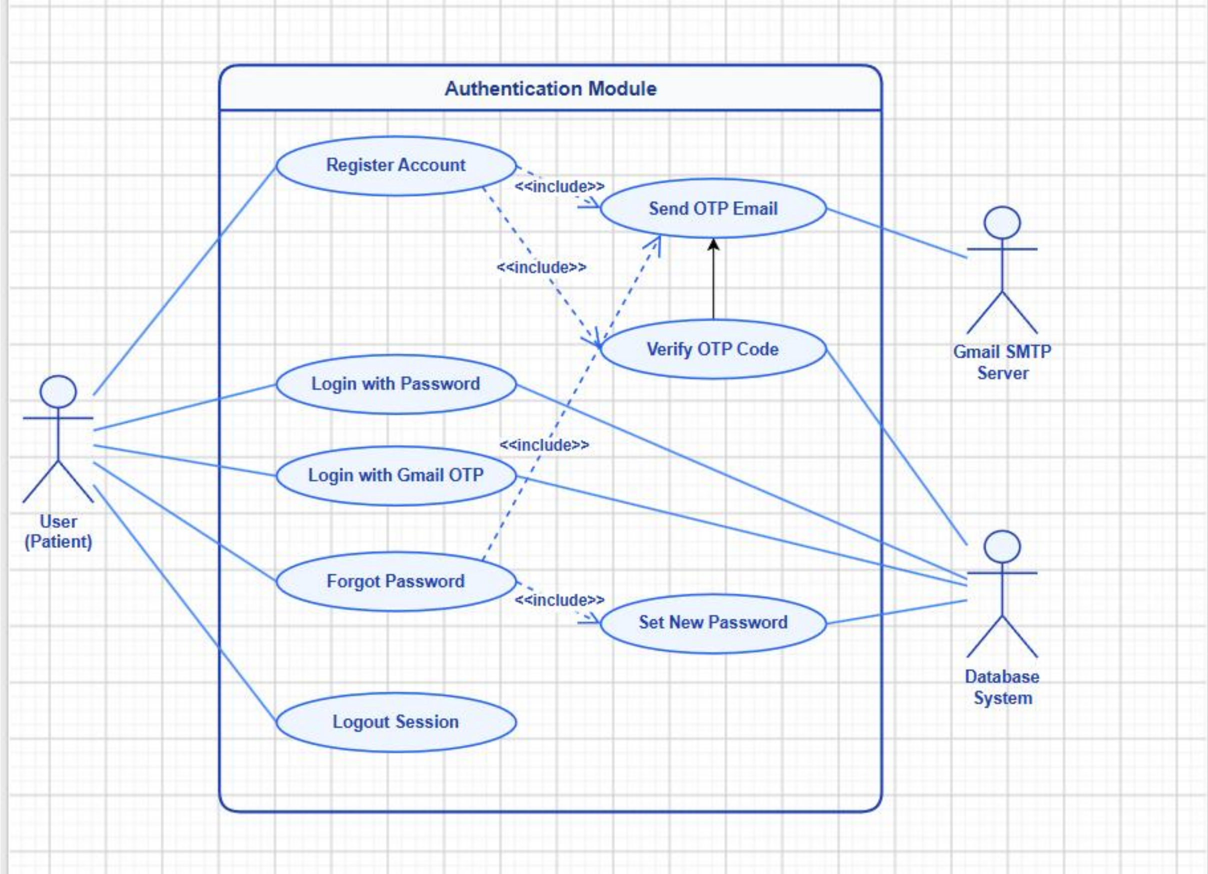


Figure 2. Authentication Use Case Diagram

- **Actor**: User.
- **Description**: Complete auth lifecycle - registration with OTP email verification, login (password or Gmail OTP), password recovery, logout.
- **Regester Flow**:
    - User enters name, email, password → System validates & checks duplicates.
    - System generates 6-digit OTP → Gmail SMTP sends to user's email.
    - User enters OTP → System validates (correct + not expired).
    - Account created → Redirect to Screening.
- **Login Flow**:
    - User enters email + password → System verifies → Redirect to Home.
    - (Alternative) Gmail OTP login: enter email → receive OTP → verify.
- **Forgot Password Flow**:
    - Enter email → Receive OTP → Verify → Set new password → Updated.

3.1.3. Health Monitoring Use Cases

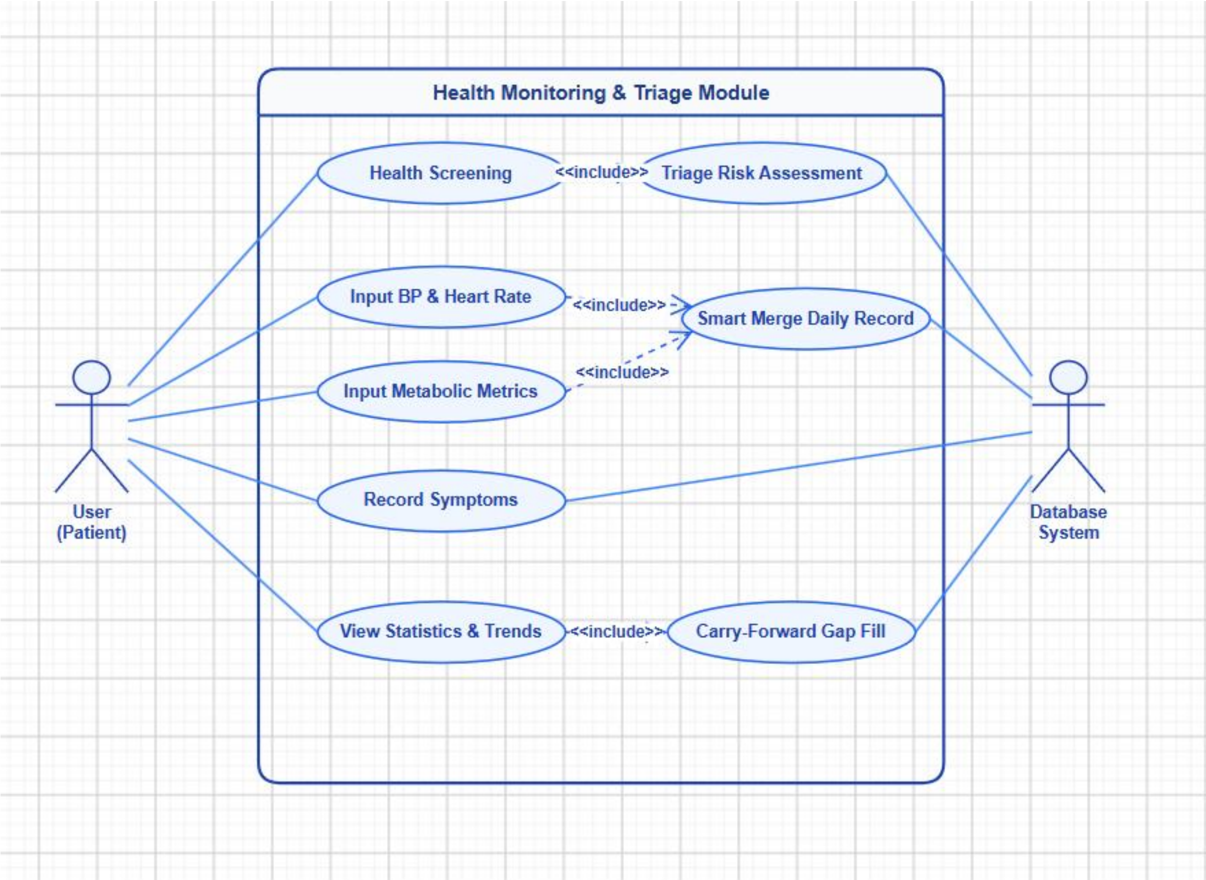


Figure 3. Health Monitoring Use Case Diagram

- **Actor**: User.
- **Flow**:
  - Health Screening & Triage - Actor: User (first-time).
    - Flow: 4-step wizard (Personal Info → Health Metrics → Medical History → Symptoms) → Backend evaluates against clinical thresholds → Returns risk level (Normal / Moderate / High / Emergency) with recommendations.
  - Input Daily Metrics - Actor: User.
    - Flow: Select category (BP/HR or Metabolic) → Adjust sliders → Save. Backend "smart merge": checks if today's record exists, merges new values without overwriting previously saved fields from other screens.
  - View Statistics - Actor: User.
    - Flow: Select period (7/30 days) → Backend retrieves metrics, groups by date, applies carry-forward for missing values → Display summary cards.

3.1.4. Scheduling & Profile Use Cases

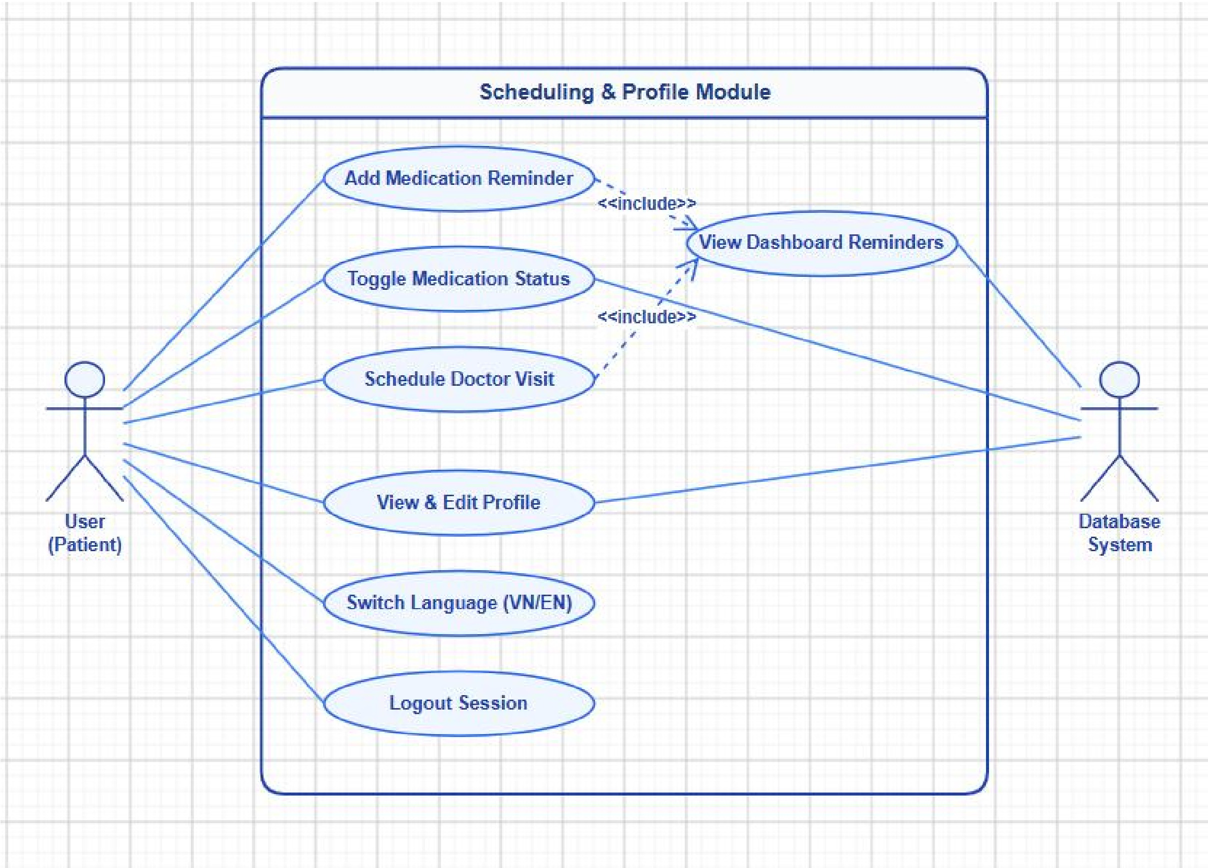


Figure 4. Scheduling Use Case Diagram

- **Actor**: User.
- **Flow**:
  - Medication Reminders - Actor: User.
    - Flow: Enter medication name, dosage, time → Save → View daily list → Toggle taken/untaken status. Also shown on Home Dashboard.The system displays the current hierarchy of levels and associated lessons.
  - Doctor Appointments - Actor: User.
    - Flow: Enter doctor name, hospital, date, time → Save → View upcoming appointments chronologically.
  - Profile Management -Actor: User.
    - Flow: View personal info, medical history, medications → Edit name, age, gender → Save → Switch language (VN/EN) → Logout.

3.1.5. Activity Diagrams

❖ Authentication

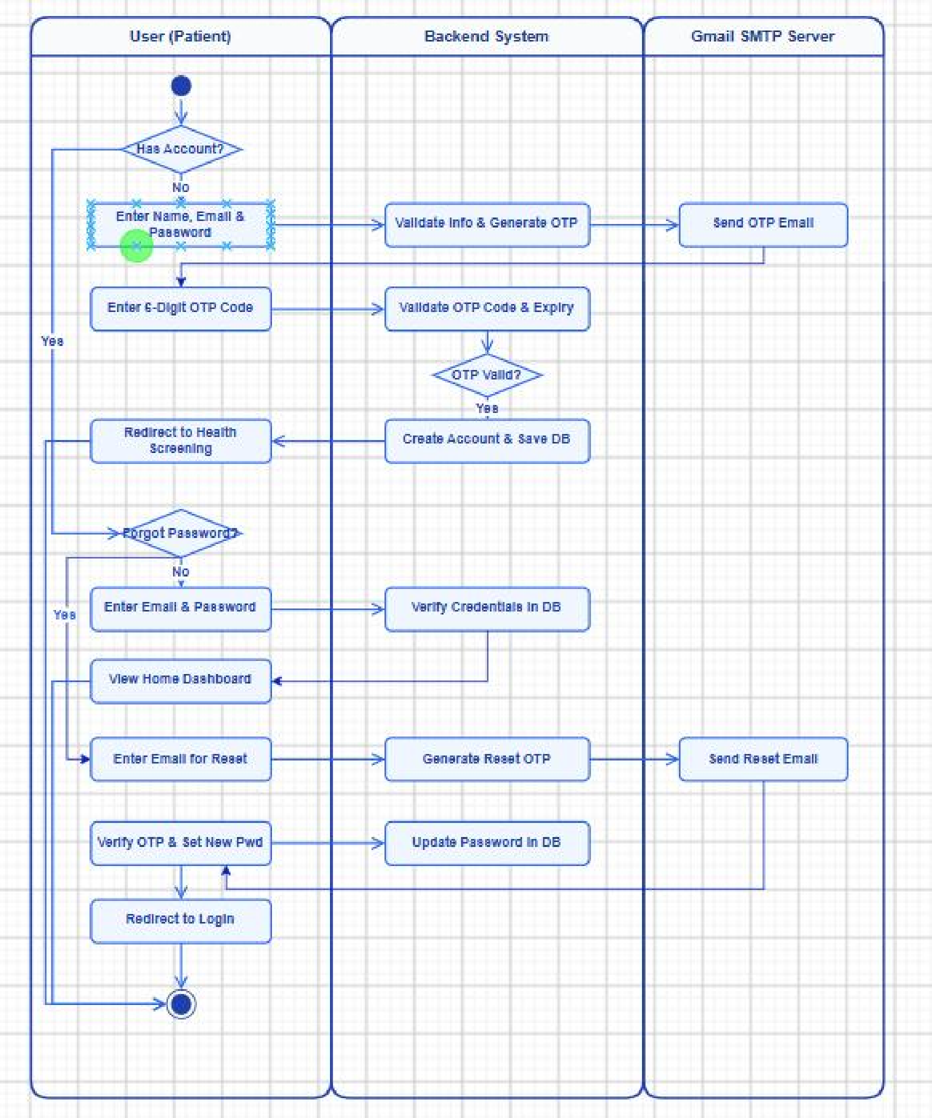


Figure 5. Authentication Activity Diagram

- **Participants**: User,System, Gmail SMTP.
- **Process Flow**:

  Start → Has Account? → No (Register): Enter info → Validate email → Generate OTP → Send via SMTP → User enters OTP → Validate → Valid: Create account → Go to Screening Invalid: Show error → Resend option → Yes: Forgot Password? → No: Enter credentials → Verify → View Home → Yes: Enter email → Send OTP → Verify → Enter new password → Update DB → Redirect to Login → End

❖ **Health Screening**

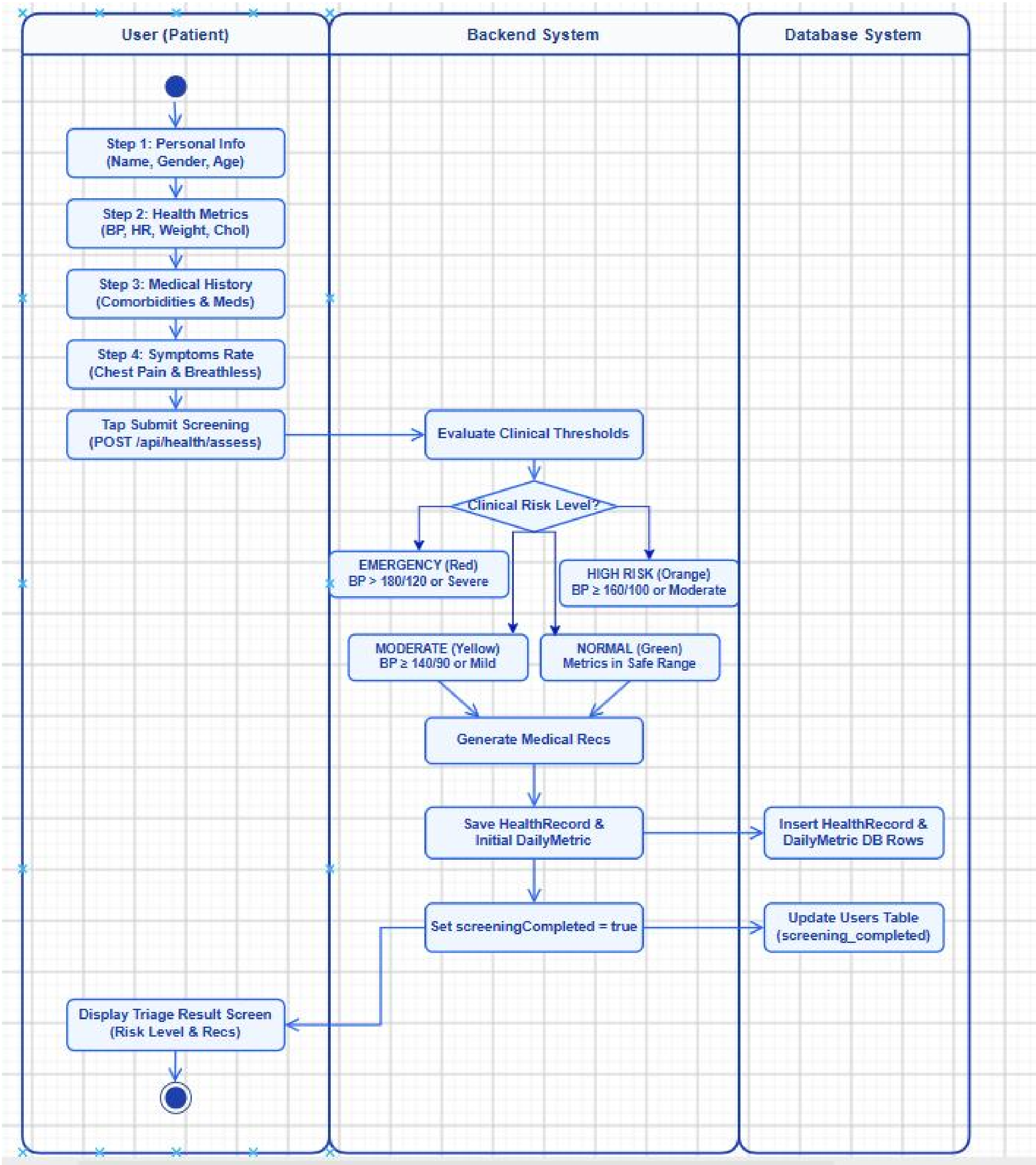


Figure 6. Health Screening Activity Diagram

- **Participants**: User, System, Database.
- **Process Flow**:

1. Start: User initiates the initial Health Screening process upon completing registration.
2. Step 1 (Personal Info): User inputs full name, gender, and age via custom slider.
3. Step 2 (Health Metrics): User records baseline vitals (blood pressure, heart rate, weight, cholesterol).
4. Step 3 (Medical History): User selects pre-existing comorbidities and inputs daily medications.
5. Step 4 (Symptoms Rating): User rates chest pain and breathlessness severity using visual icons.
6. Submit to API: App transmits payload to POST /api/health/assess.
7. Evaluate Thresholds: Backend evaluates vital signs against medical guidelines:
    - EMERGENCY (Red): BP > 180/120 mmHg OR severe clinical symptoms.
    - HIGH RISK (Orange): BP ≥ 160/100 mmHg OR moderate symptoms.
    - MODERATE (Yellow): BP ≥ 140/90 mmHg OR mild indicators.
    - NORMAL (Green): All metrics remain within safe medical thresholds.
8. Save to DB: System persists HealthRecord and initial DailyMetric entries to MySQL database.
9. Update User State: System sets screeningCompleted = true in user record. Display Triage Result: App displays risk level, summary assessment, and 4 medical recommendations.
10. End: User completes screening and navigates to Main Home Dashboard.

❖ Daily Metrics

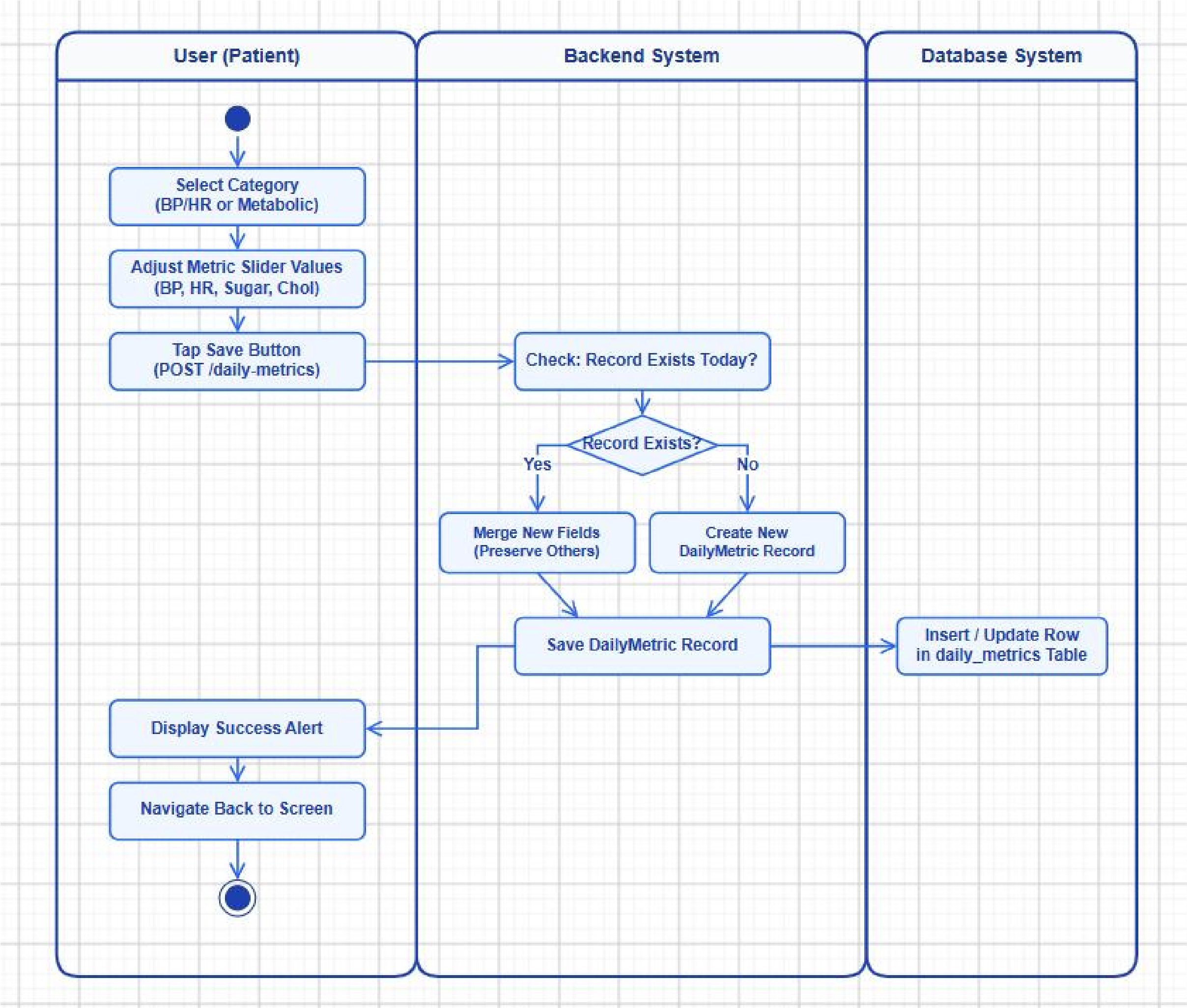


Figure 7. Daily Metrics Activity Diagram

- **Participants**: User, System, Database.
- **Process Flow**:

  Start → Select category (BP/HR or Metabolic) → Adjust slider values → Submit to POST /api/health/daily-metrics → Check: record exists for today? → Yes: Merge new fields into existing record (preserve others) → No: Create new DailyMetric record → Save to DB → Show success alert → Navigate back → End

❖ Scheduling

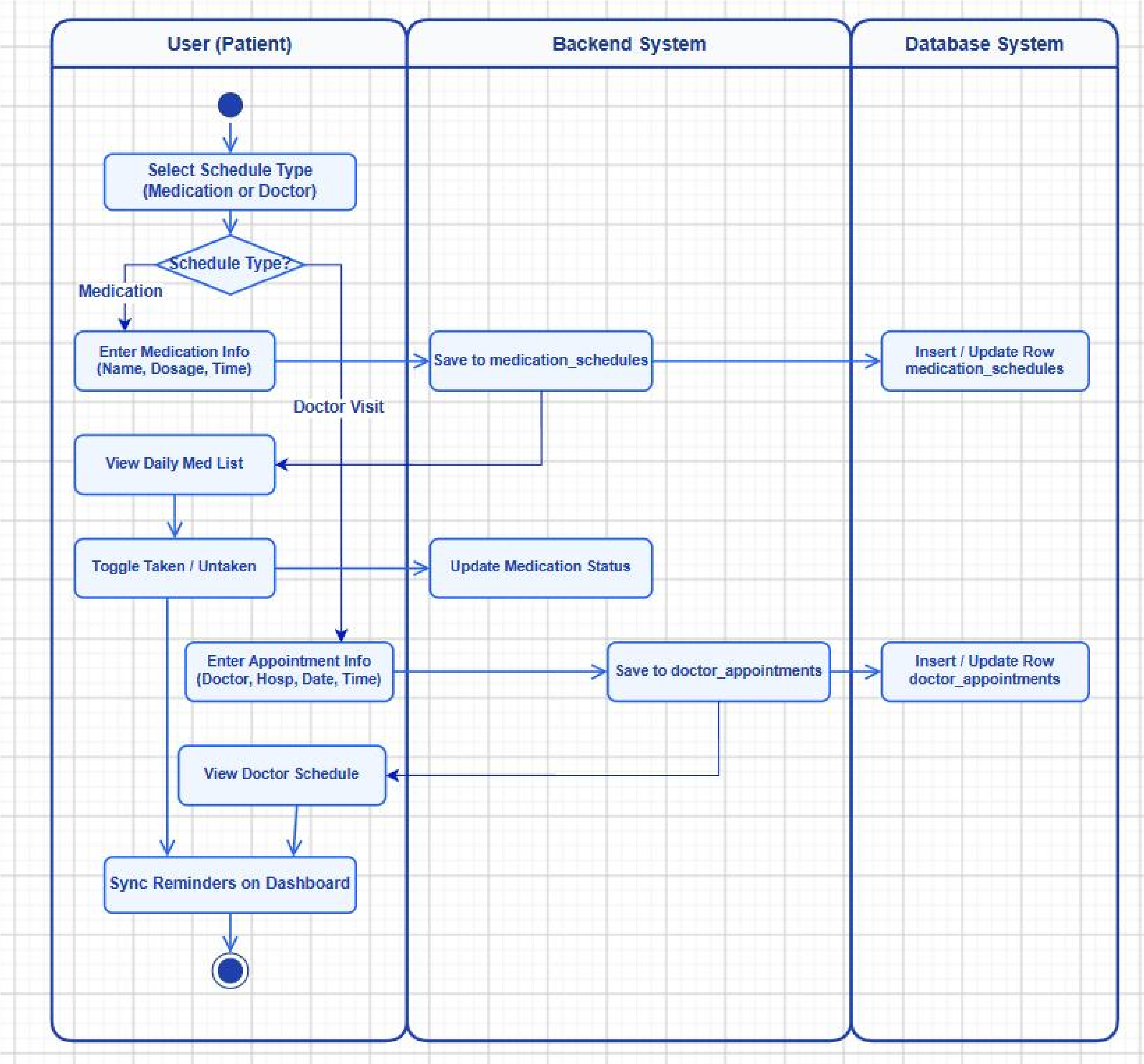


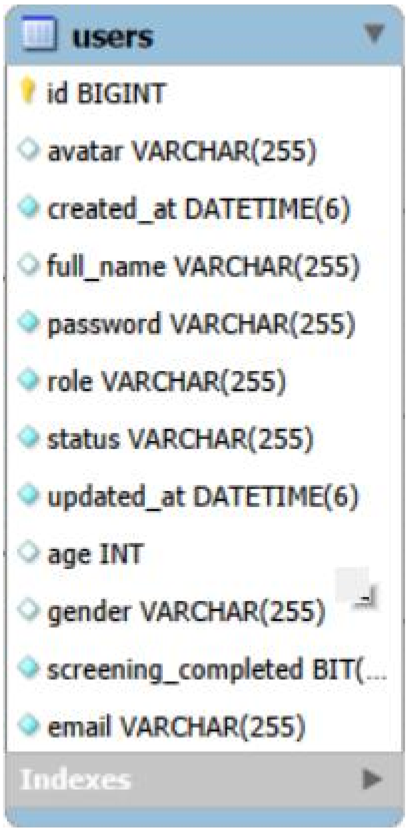
Figure 8. Scheduling Activity Diagram

- **Participants**: User, System, Database..
- **Process Flow**:

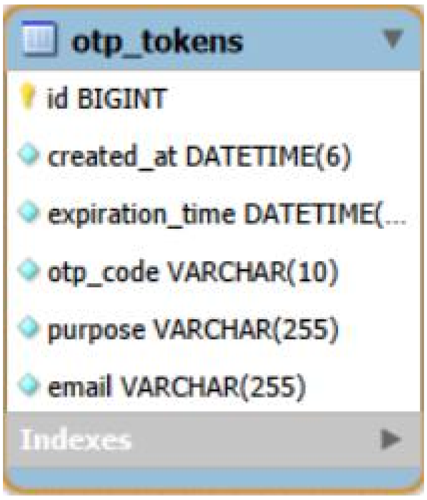
  Start → Select type (Medication or Doctor) → Medication: Enter name, dosage, time → Save to medication_schedules → View list → Toggle taken/untaken → Doctor: Enter doctor, hospital, date, time → Save to doctor_appointments → View upcoming list → End.

3.2. Database

3.2.1. Users Table

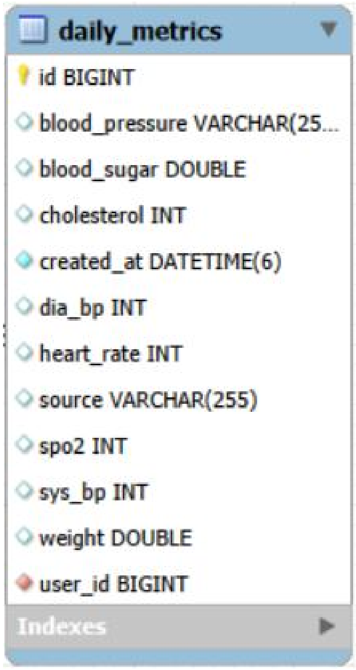


3.2.2. OTP_Tokens Table

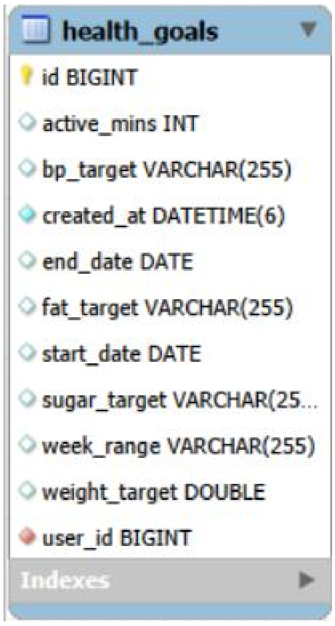


3.2.3. Health_Records Table

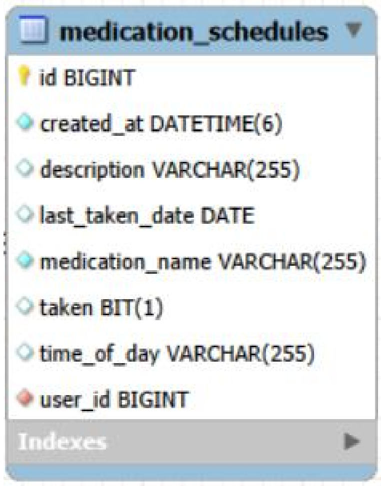
3.2.4. Daily_Metrics Table



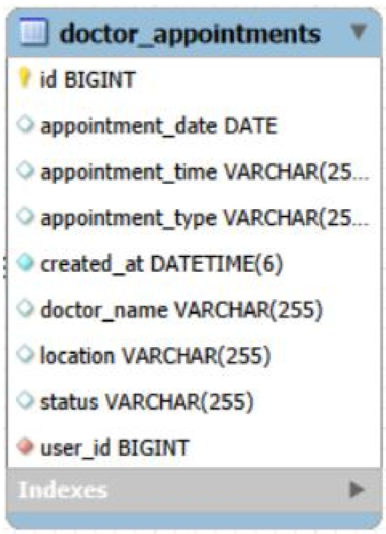
3.2.5. Health_Goals Table



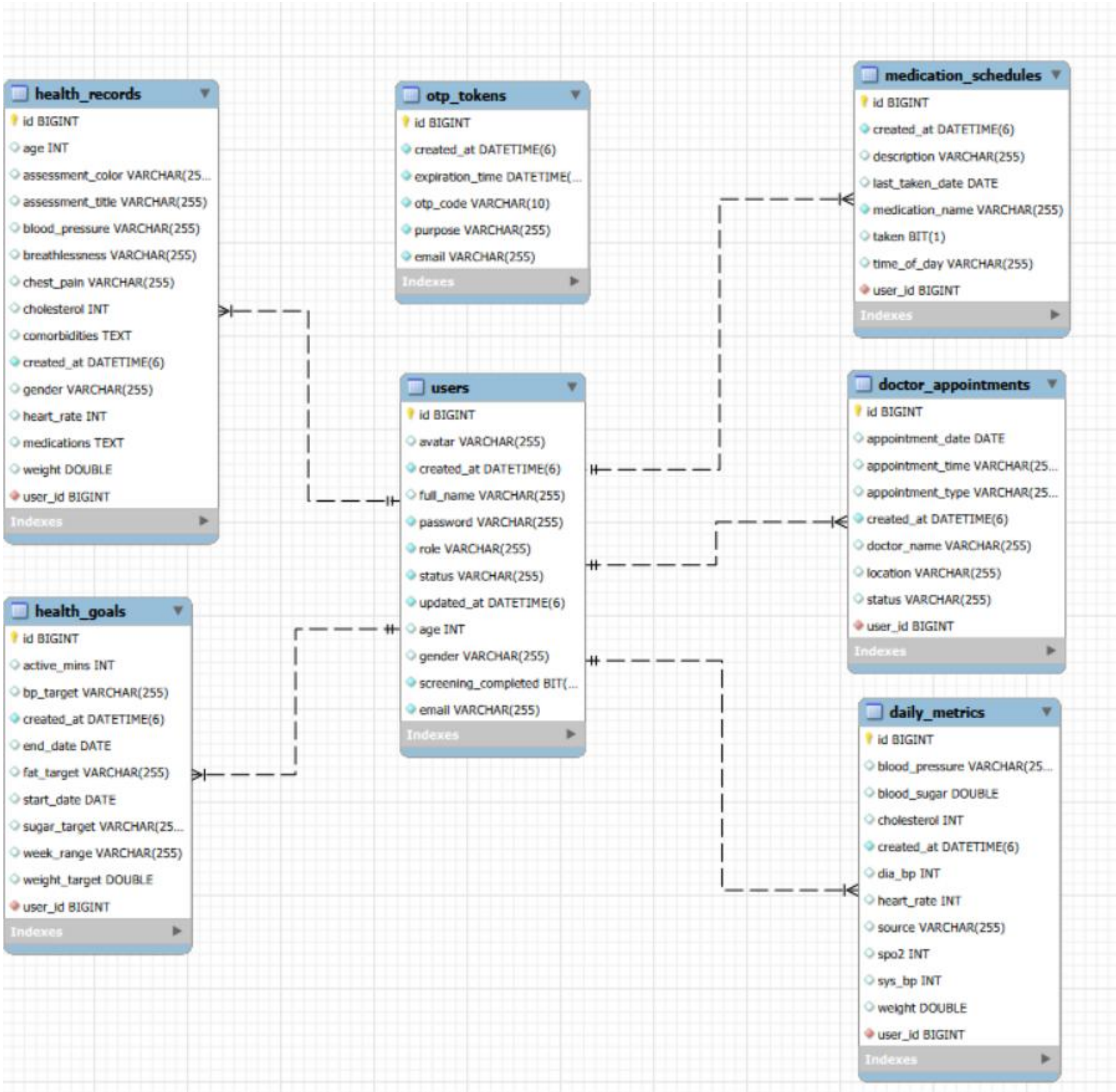
3.2.6. Medication_Schedules Table



3.2.7. Doctor_Appointments Table



3.2.8. Relationships of Database



CHAPTER 4. Application

4.1. Login and Authentication

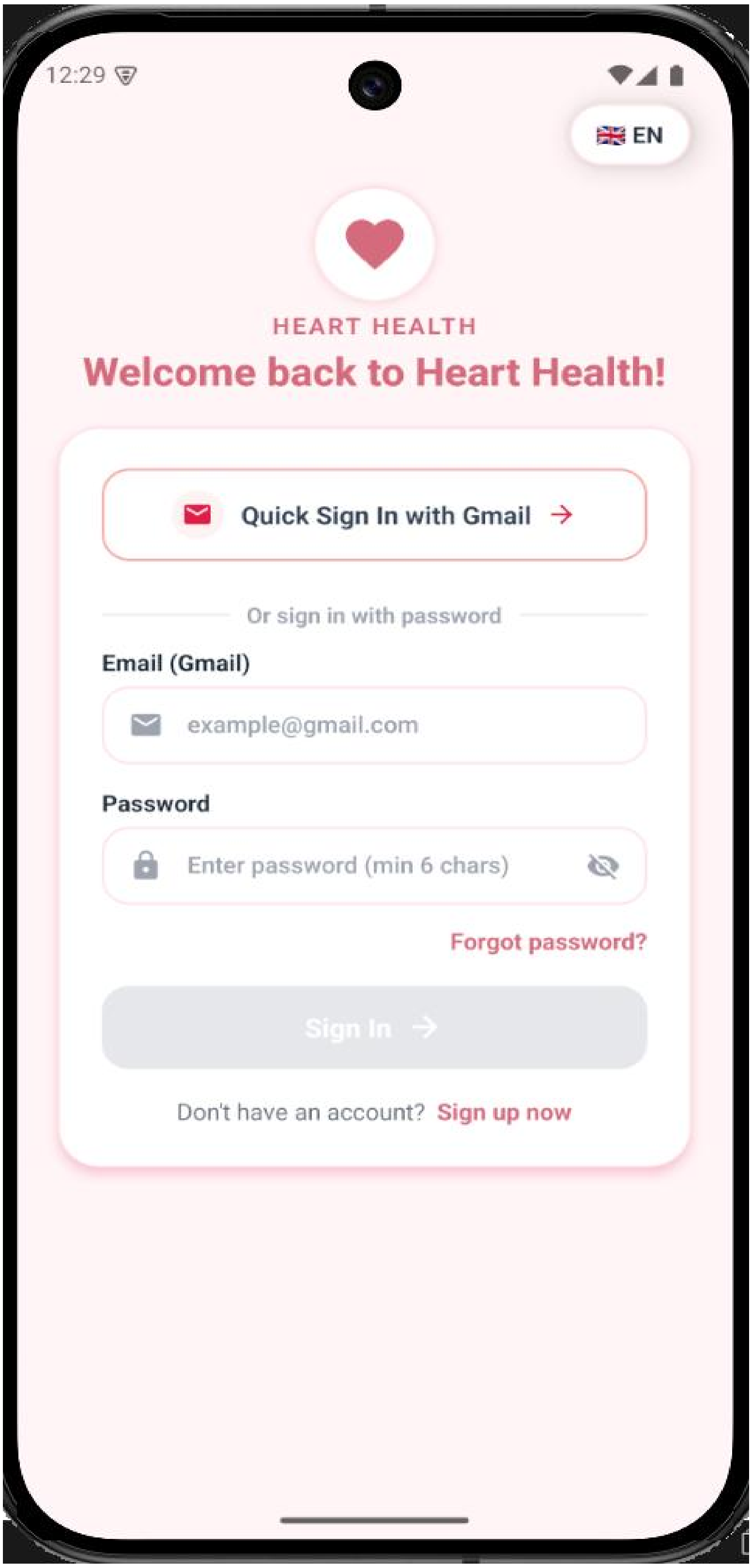


Figure 9. Login Screen

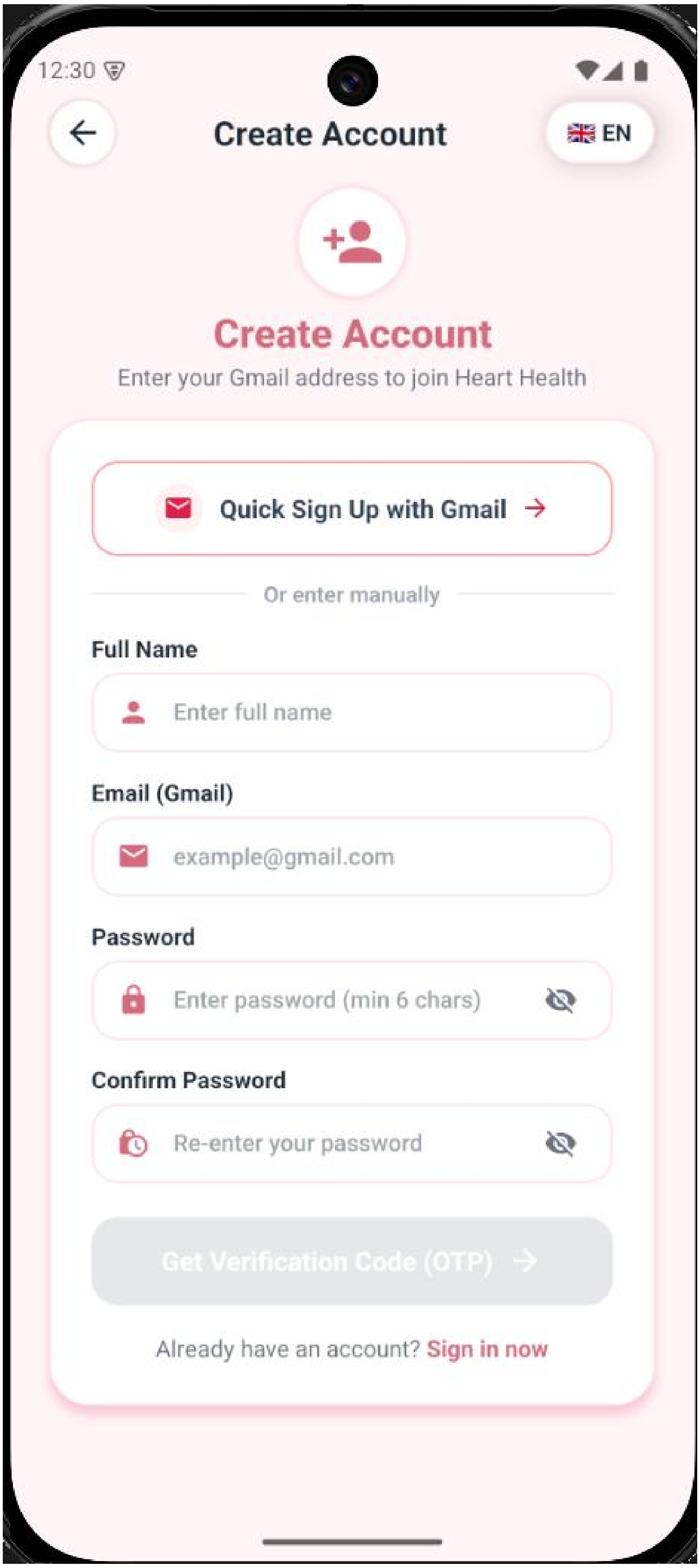


Figure 10. Registration Screen

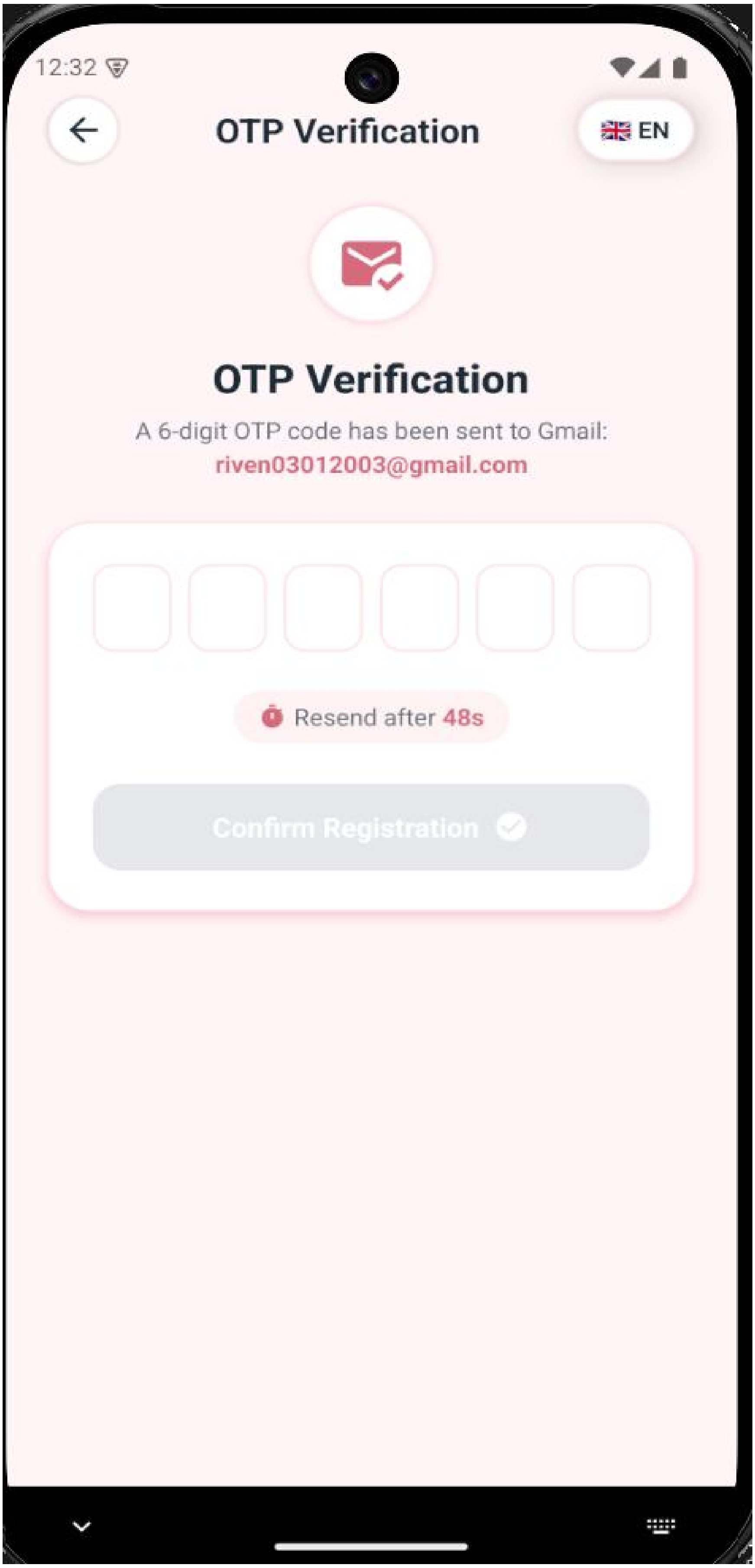


Figure 11. OTP Verification Screen

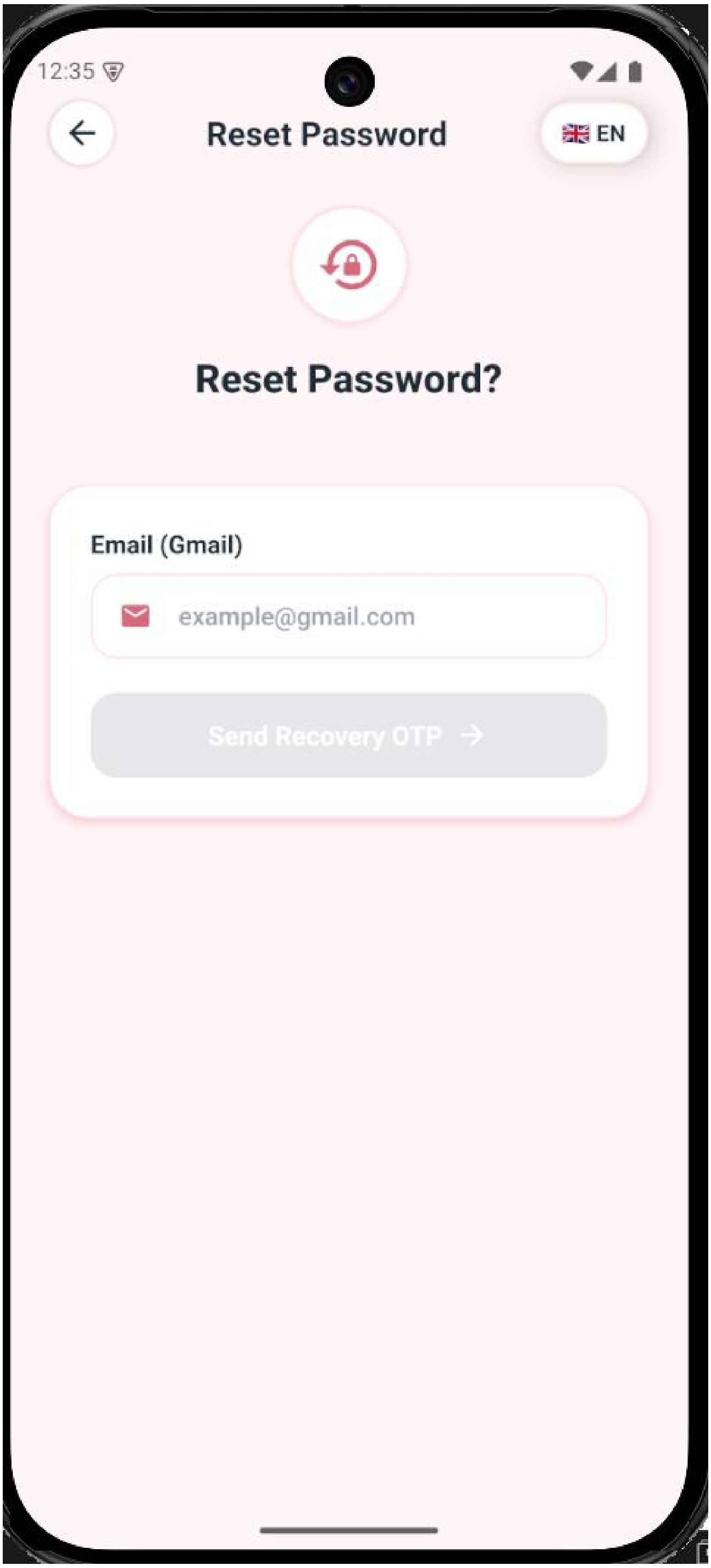


Figure 12. Password Reset Interface

4.2. Health Screening & Triage

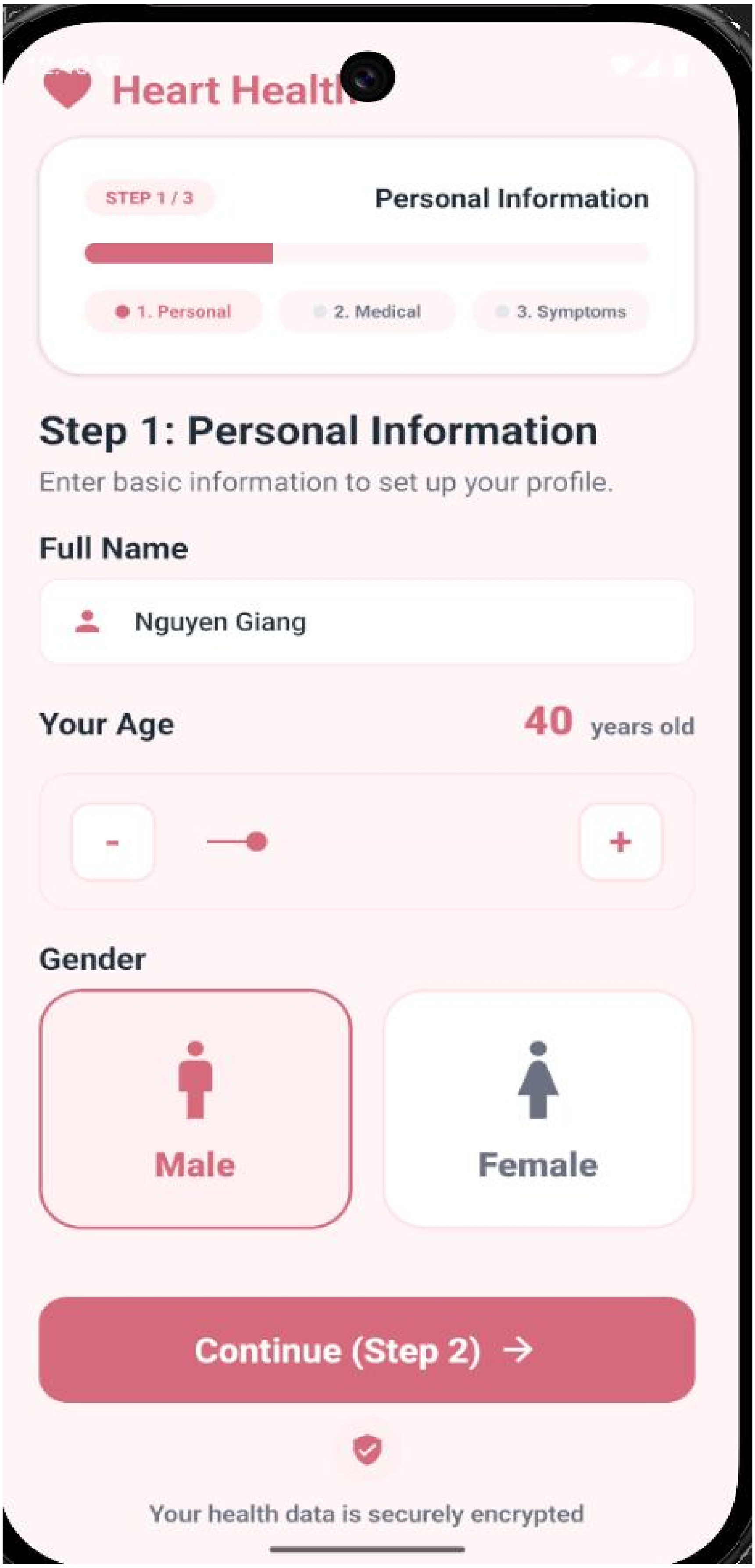


Figure 13. Screening Step 1

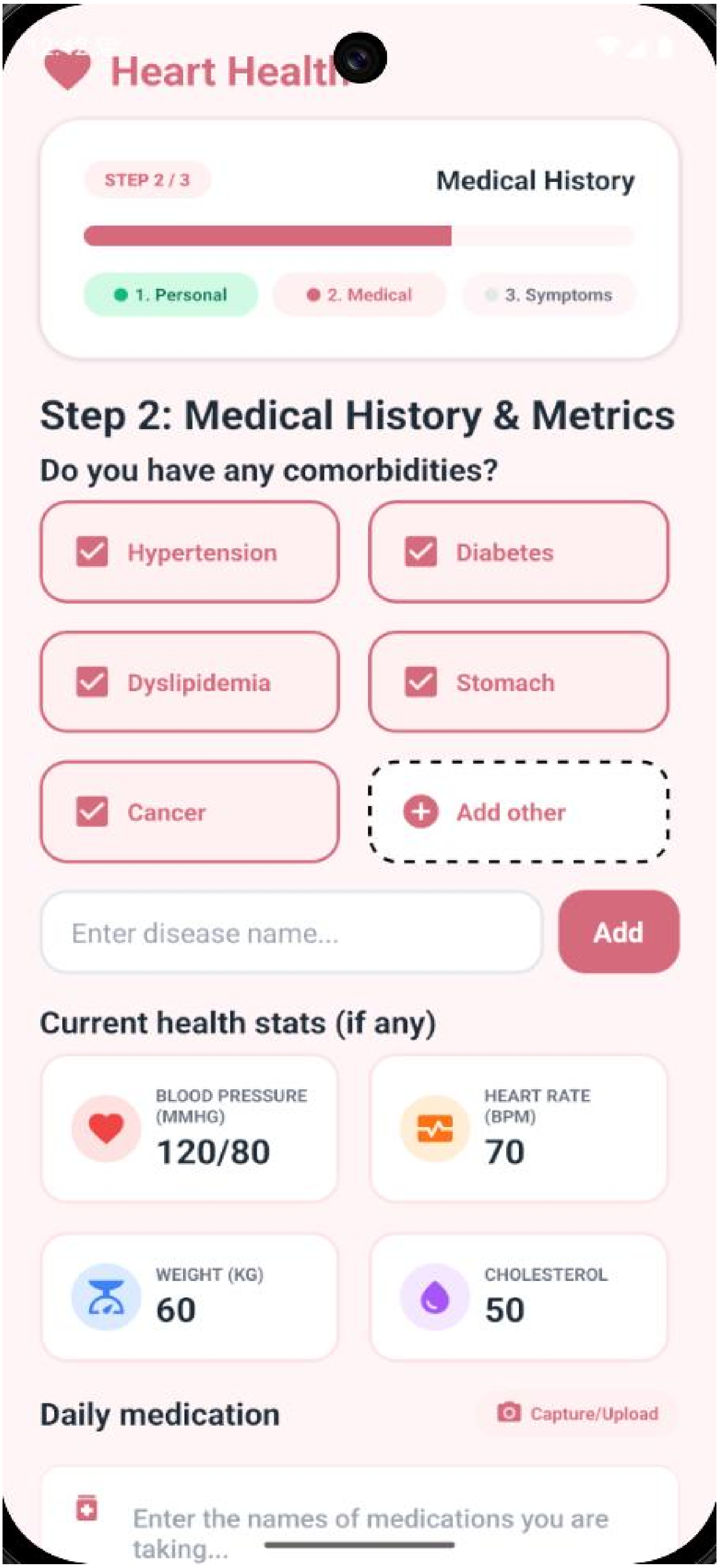


Figure 14. Screening Step 2

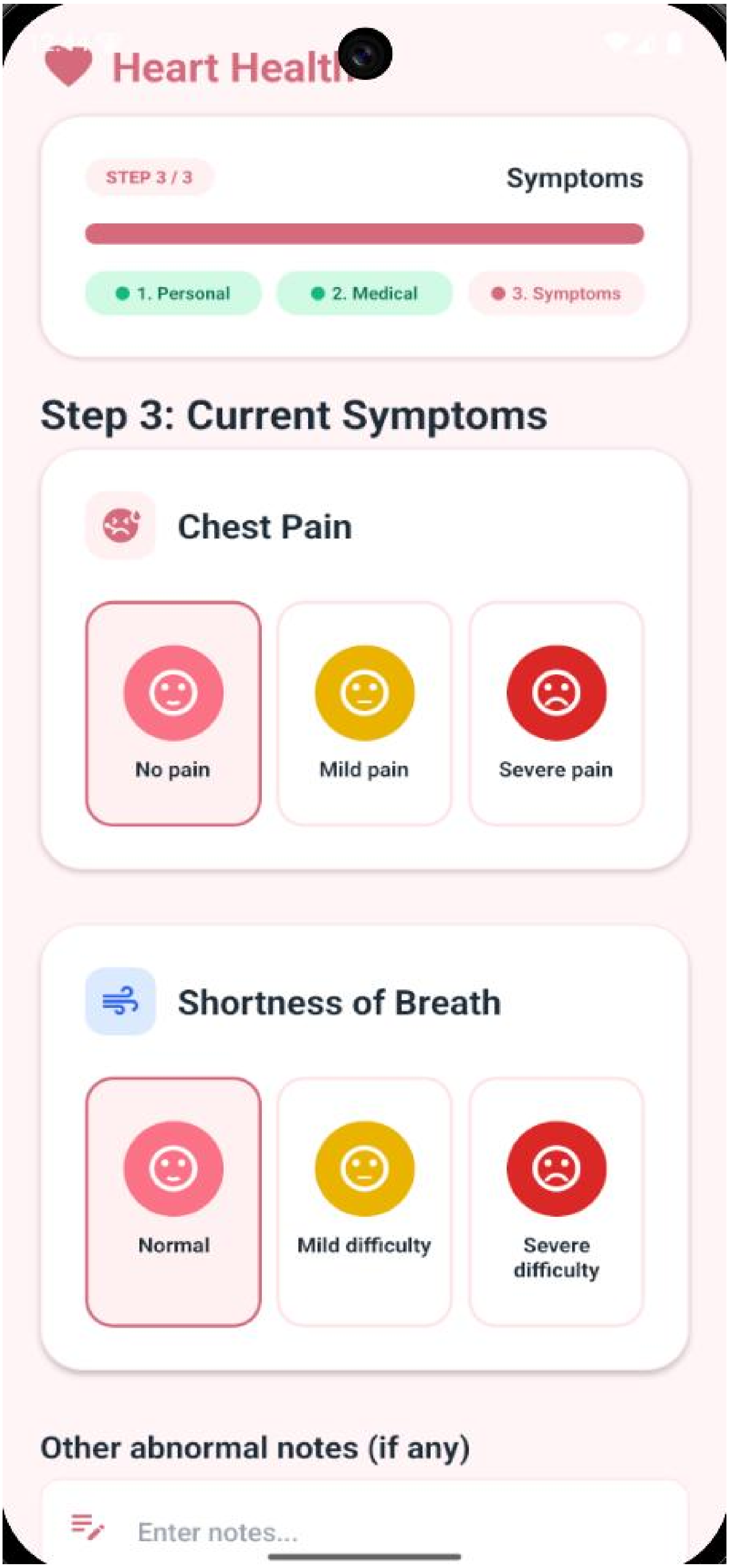


Figure 15. Screening Step 3

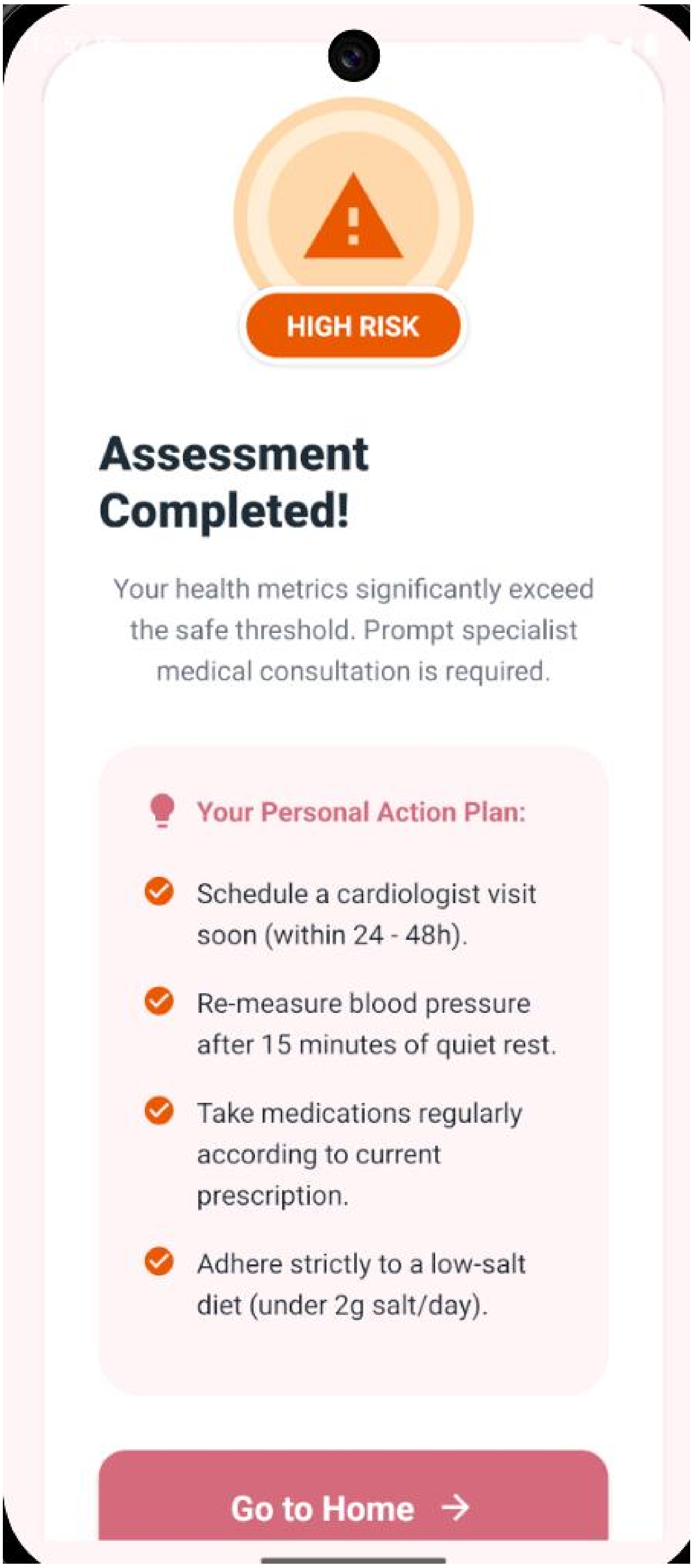
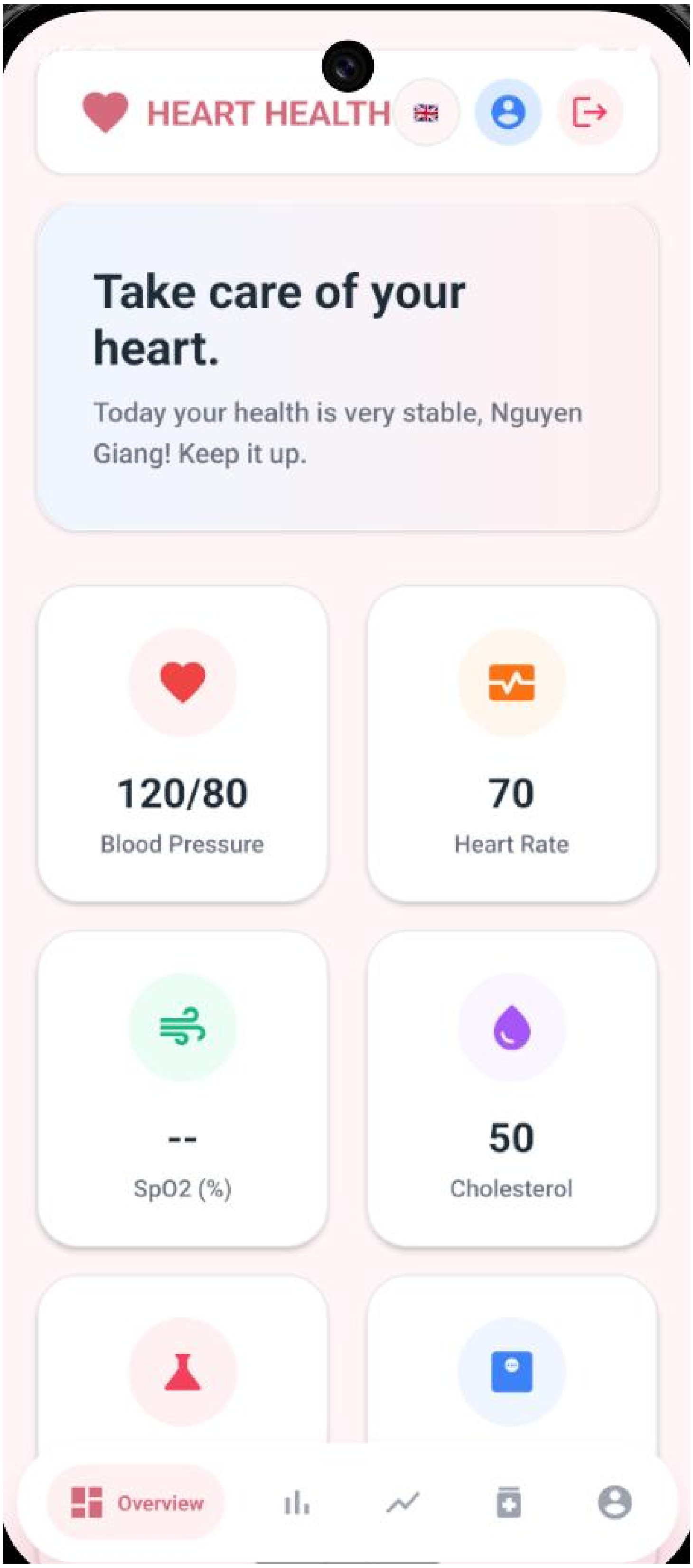
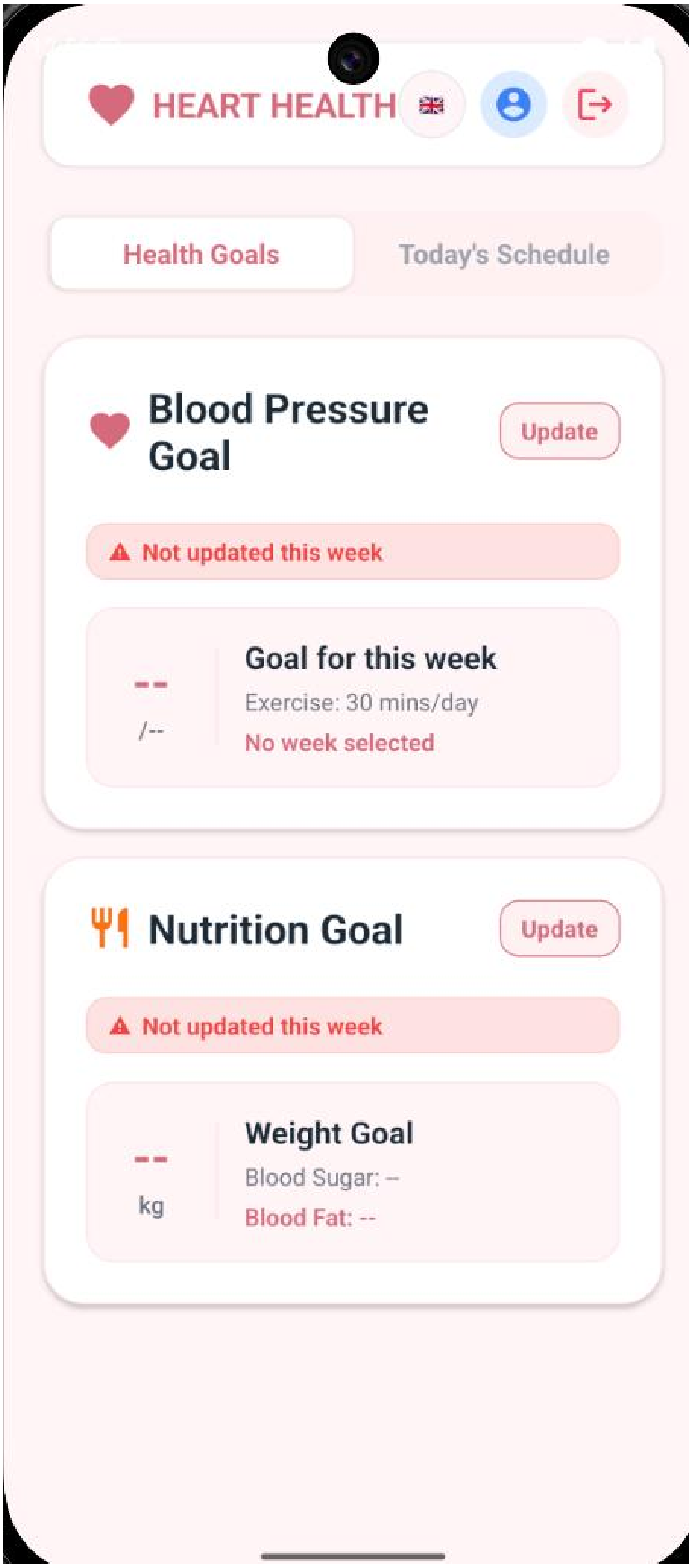


Figure 16. Triage Result Screen

4.3. Main Dashboard




HEART HEALTH
Health Goals
Today's Schedule
Blood Pressure Goal
Update
Not updated this week
--
/--
Goal for this week
Exercise: 30 mins/day
No week selected
Nutrition Goal
Update
Not updated this week
--
kg
Weight Goal
Blood Sugar: --
Blood Fat: --

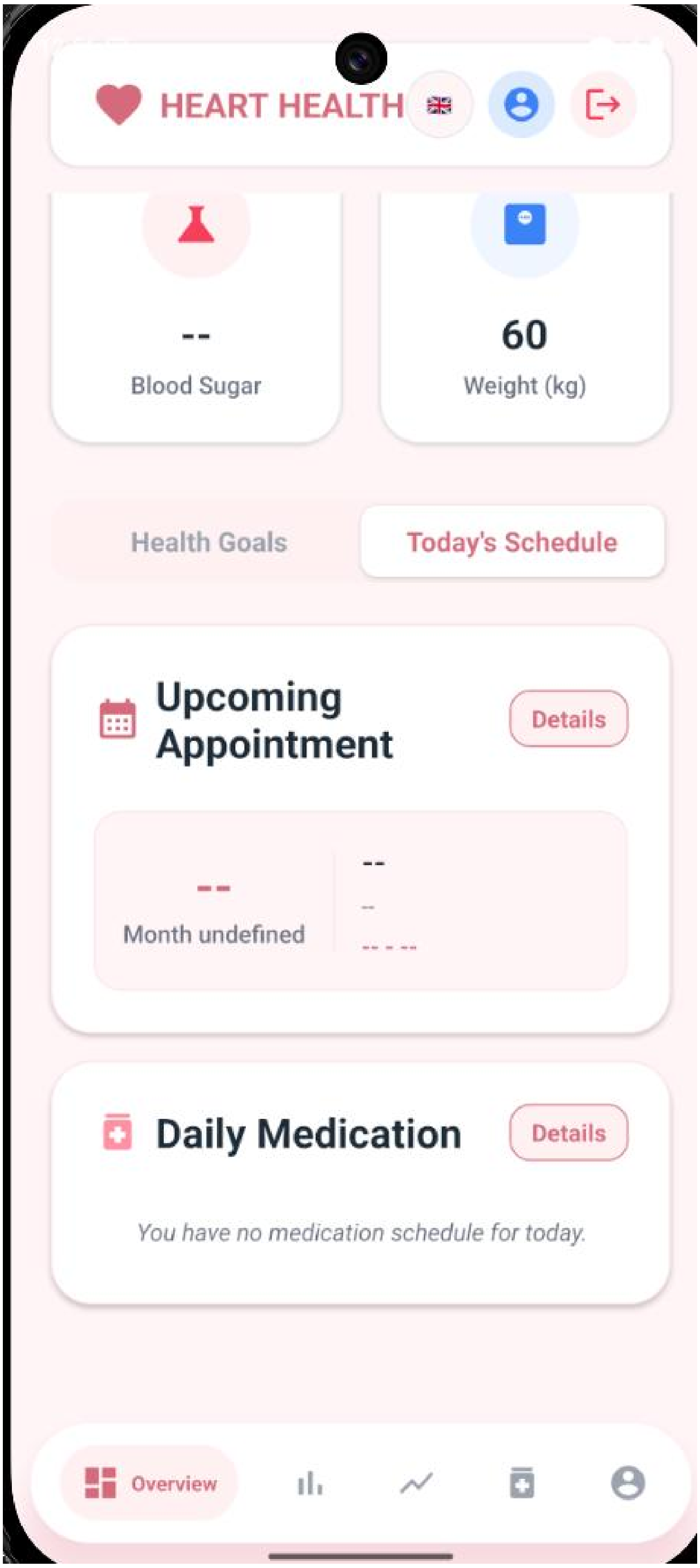


Figure 17. Home Dashboard

4.4. Daily Metrics Entry

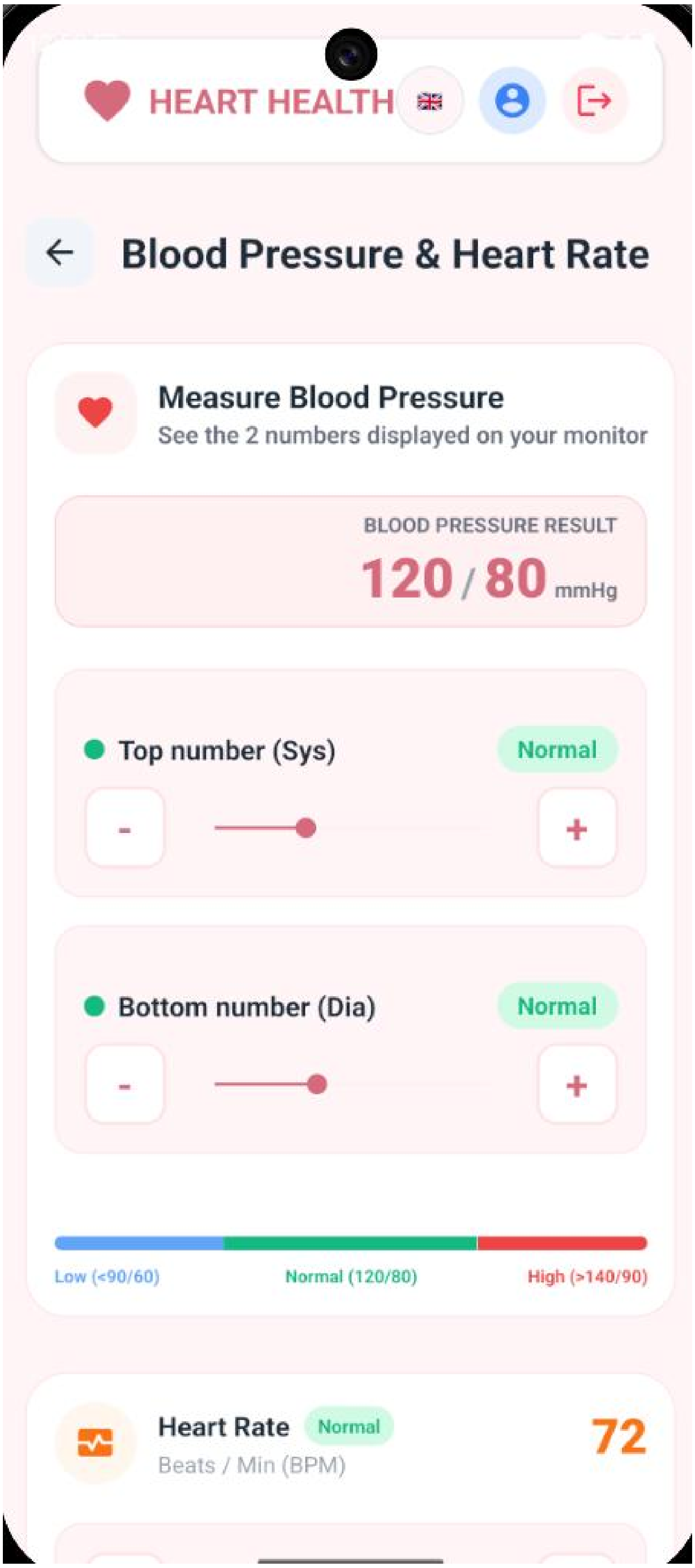


Figure 18. BP & Heart Rate Entry

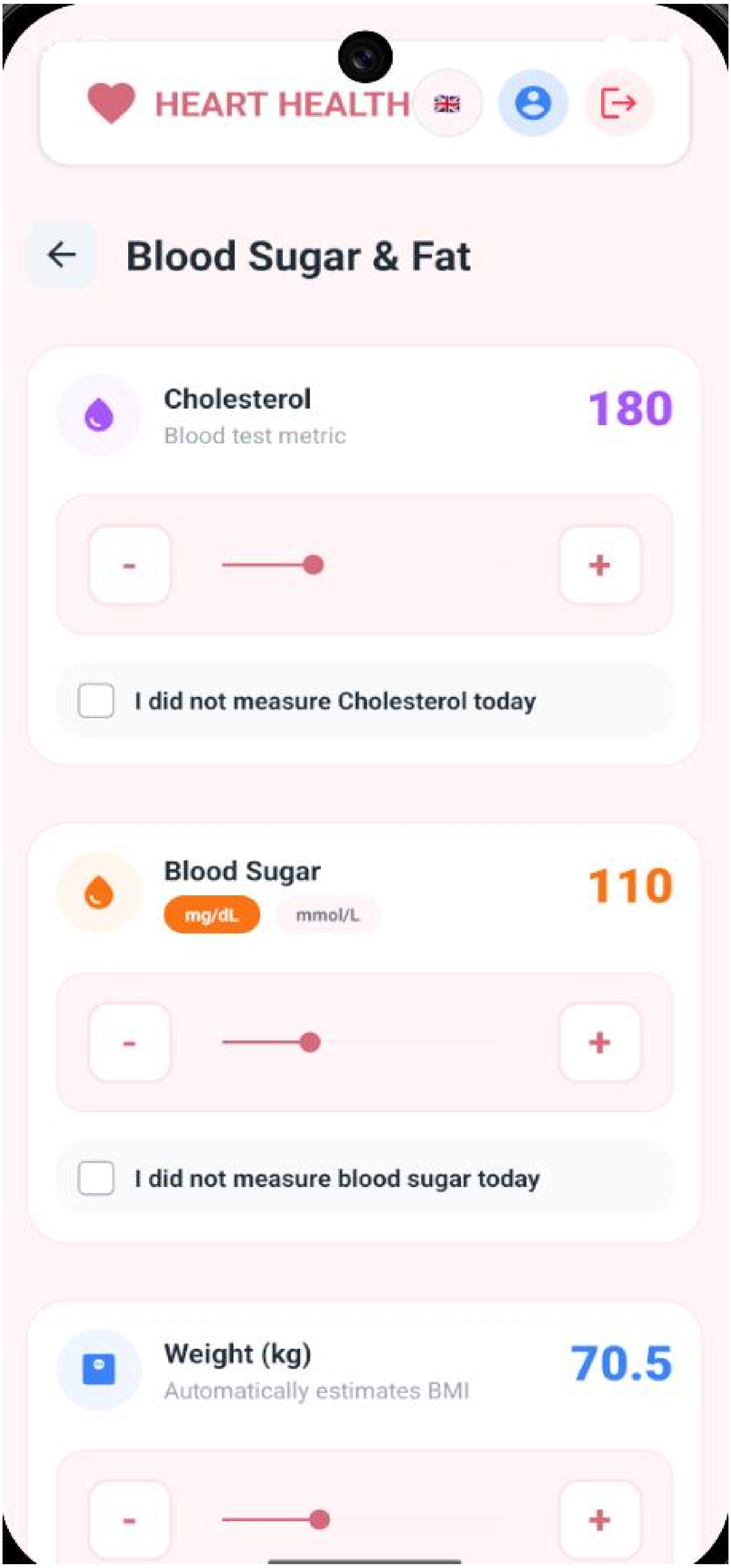


Figure 19. Metabolic Entry

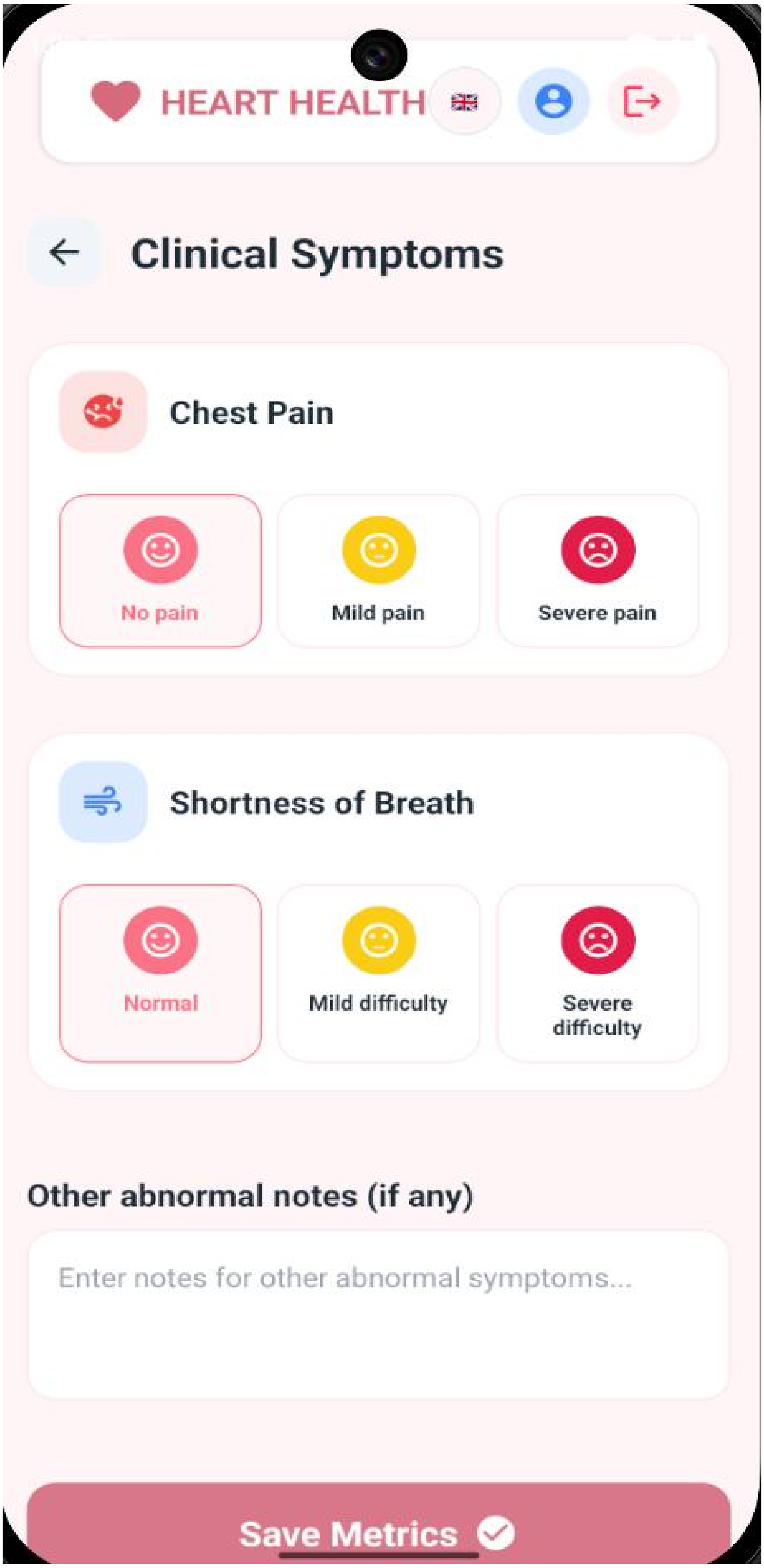


Figure 20. Symtons Entry

4.5. Statistics & Charts

Figure 21. Statistics View

4.6. Scheduling

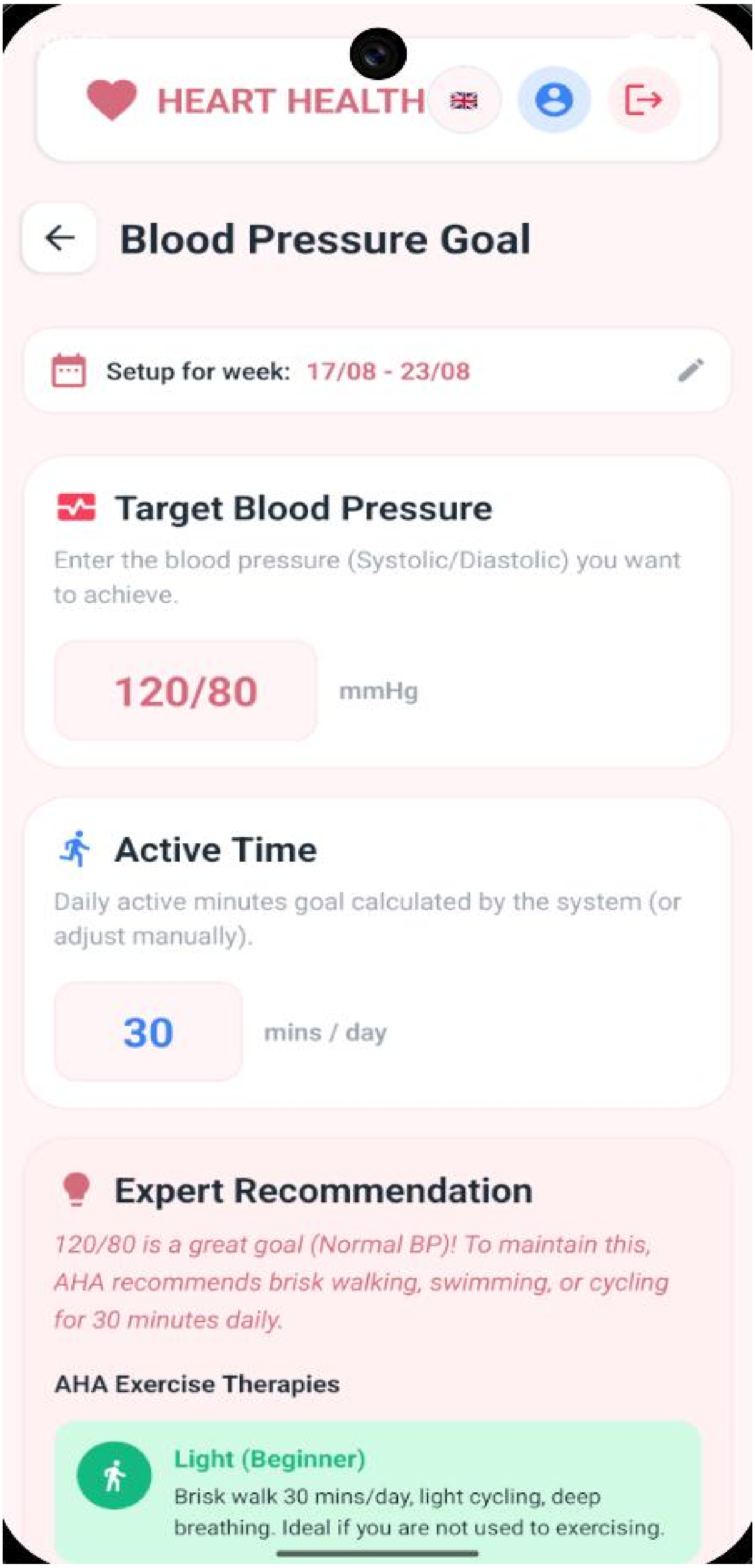


Figure 22. Blood Pressure Goal

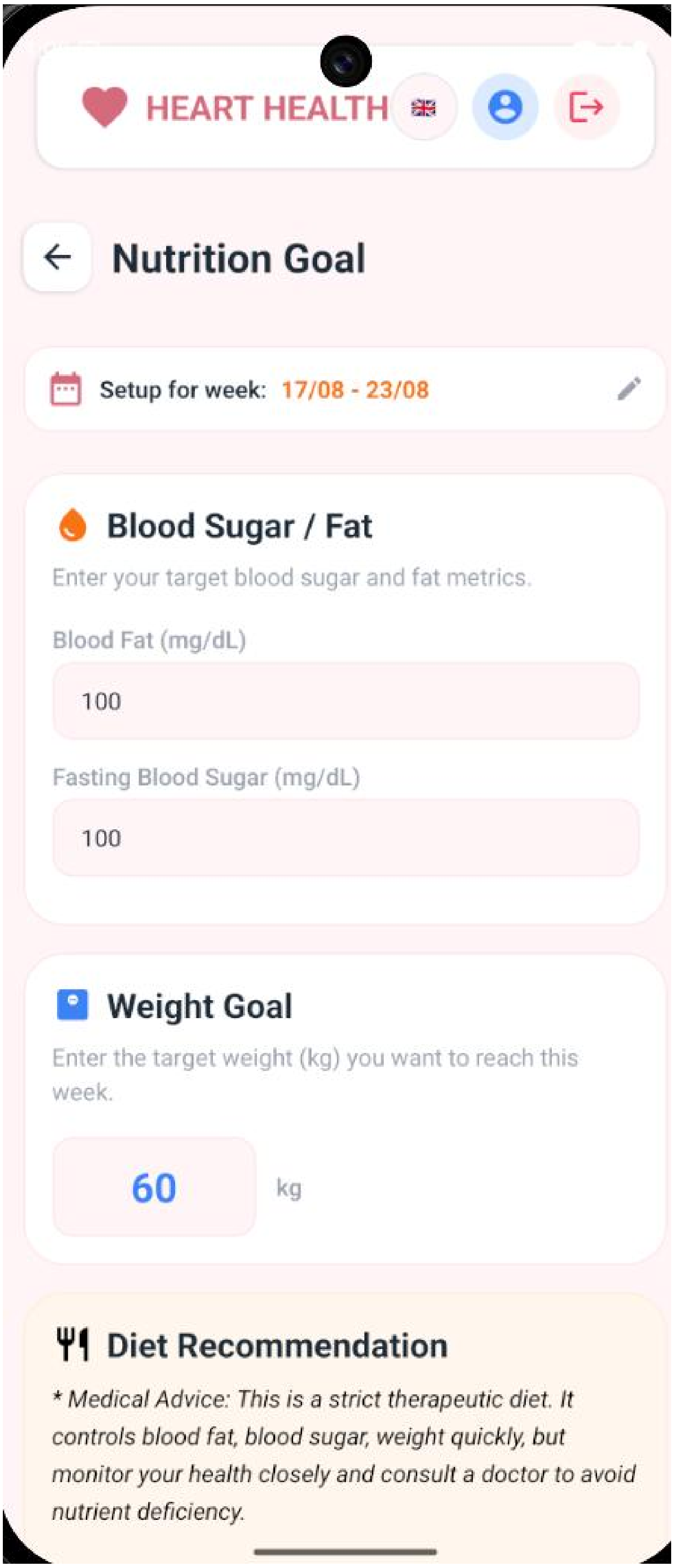


Figure 23. Nutrition Goal

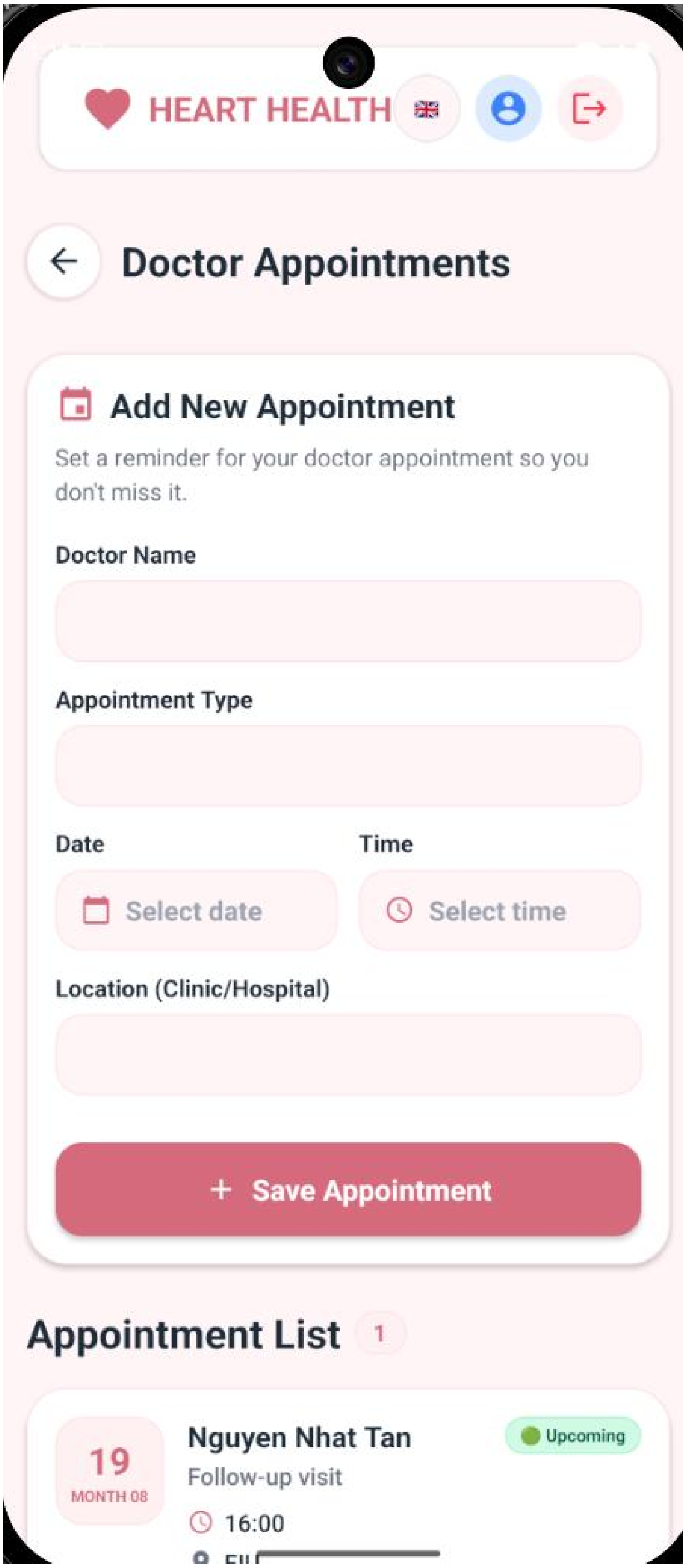


Figure 24. Doctor Appointments

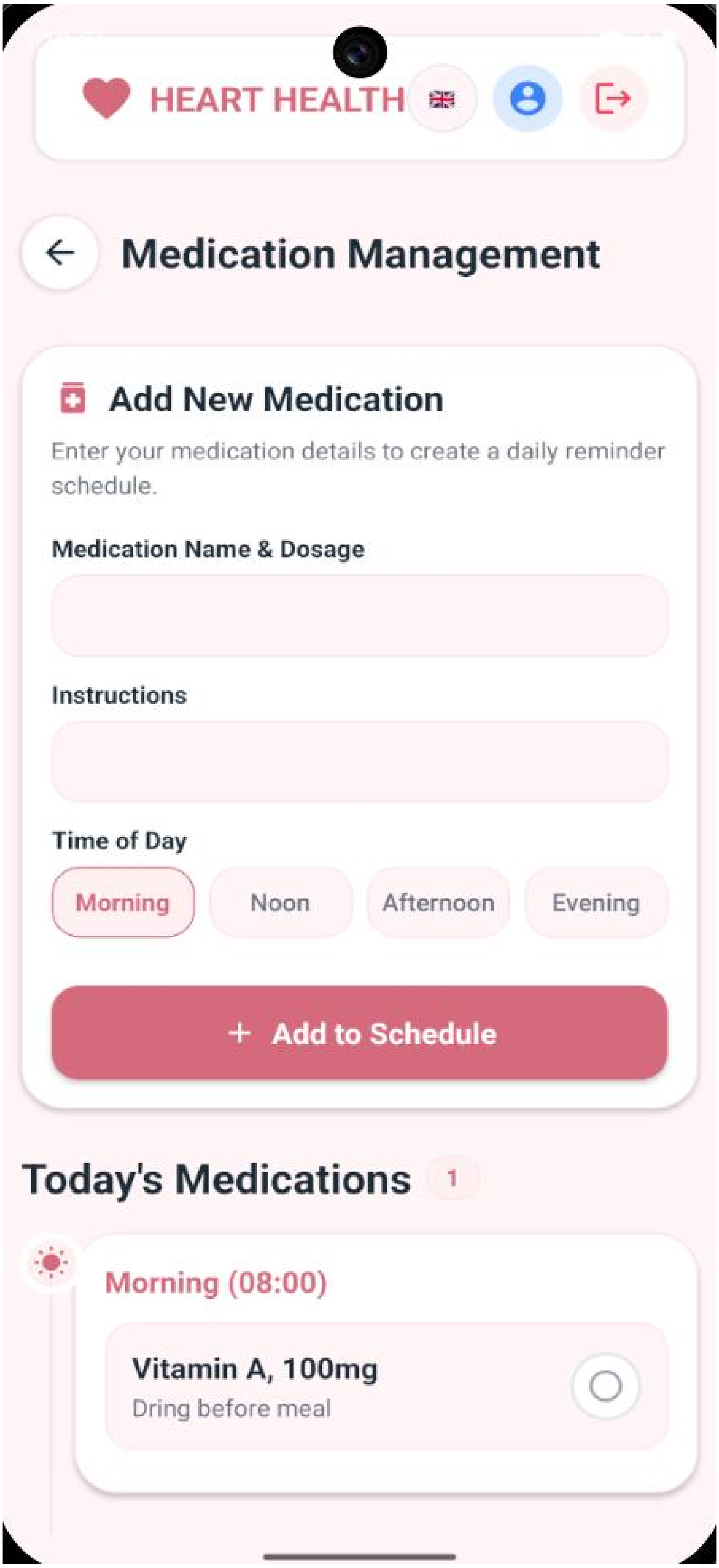


Figure 25. Medication Management

4.7. Profile

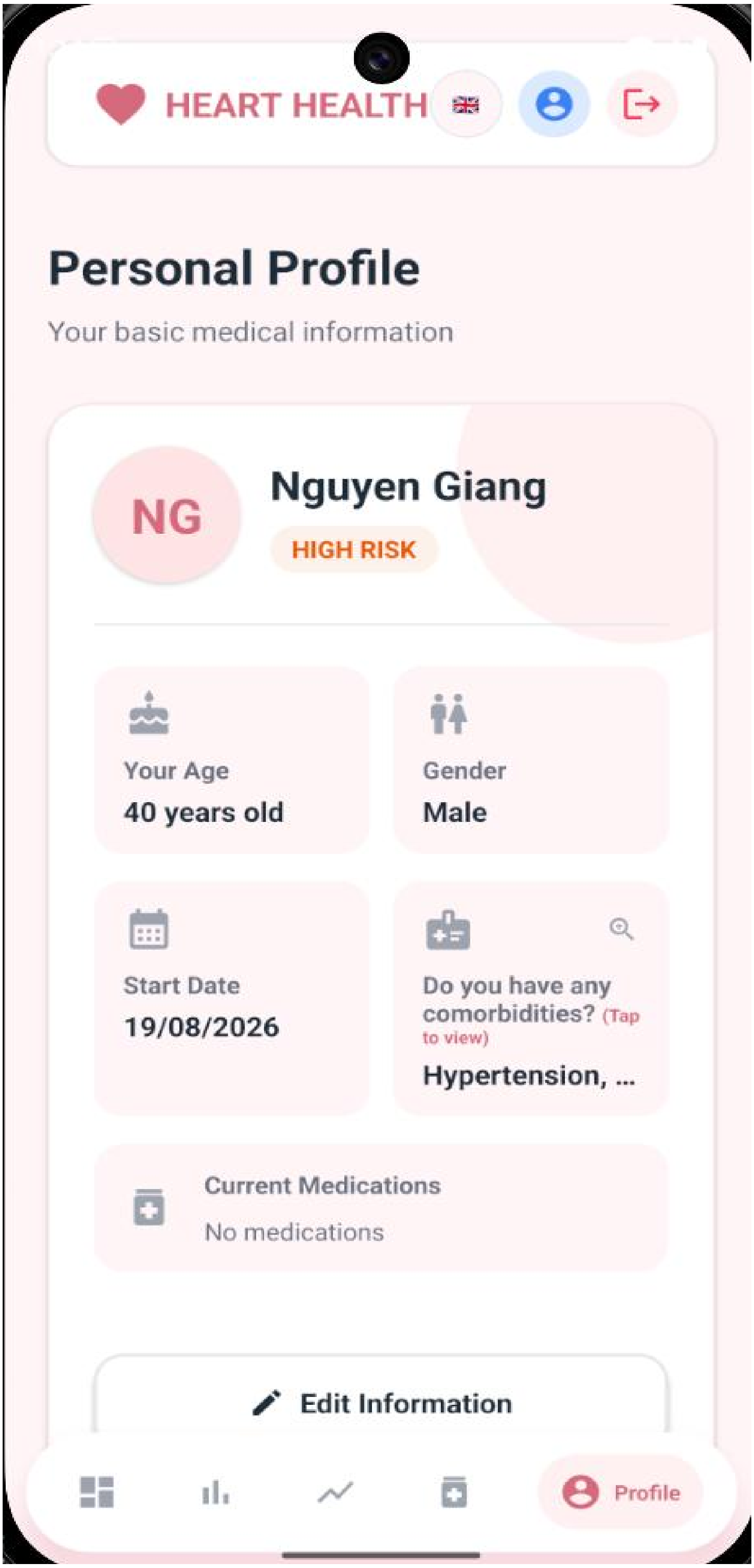


Figure 26. Profile Screen

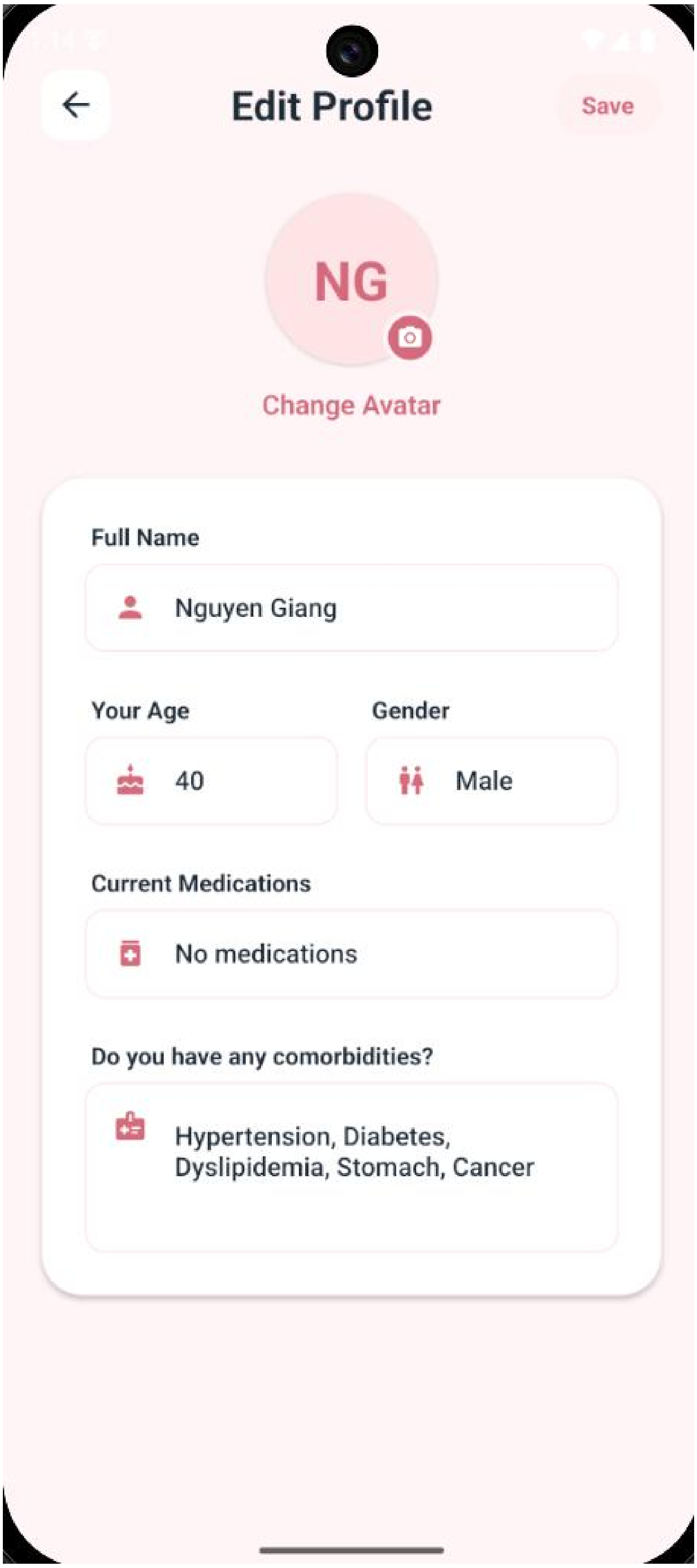


Figure 27. Edit Profile Screen

CHAPTER 5. Conclusion and Future Works

5.1. Conclusion

In this project, we successfully designed and implemented HeartHealth — a cross-platform mobile application dedicated to personal cardiovascular health tracking, risk assessment, and appointment/medication scheduling. We effectively applied the knowledge and skills acquired throughout our academic coursework, alongside the invaluable supervision and guidance provided by Mr. Trần Văn Tài and Mr. Nguyễn Hoàng Lý, to deliver a fully functional, medical-grade mobile solution tailored for patients with cardiovascular conditions.

To achieve a production-ready application, we incorporated modern, enterprise-grade technologies across the entire system stack:

On the backend side, the system was developed using Java 17 and Spring Boot 4.1.0, establishing a robust, type-safe, and asynchronous RESTful service. The architecture follows a strict layered design to enforce a clear separation of concerns. RESTful APIs were structured across four core controllers (Auth, Health, Goal/Schedule, Admin) to manage user accounts, health screening evaluations, daily metric recording, and reminders.

Security and authentication were implemented via an email-based One-Time Password (OTP) system powered by Gmail SMTP (Spring Boot Starter Mail). Cryptographically generated 6-digit tokens with automatic 5-minute expiration ensure secure registration, quick login, and password recovery. All critical business logic - including password hashing, OTP verification, and triage risk evaluation - is strictly processed server-side to guarantee data integrity and prevent client-side tampering.

On the database side, the platform utilizes MySQL 8.0+ managed through Spring Data JPA and Hibernate ORM. The relational database schema comprises seven core tables , supported by 12+ versioned DDL migration scripts. Technical innovations in the data layer include an in-place "Smart Merge" algorithm (which merges daily vital metrics without overwriting previously saved fields) and a "Carry-Forward" aggregation mechanism to fill data gaps for continuous trend line rendering.

On the frontend side, the mobile application was built using React Native, producing a native-rendered user interface supporting both Android and iOS platforms. The UI/UX features a custom Pink Rose medical theme across 20 distinct screens, utilizing react-navigation for smooth tab and stack transitions, react-native-community/slider for intuitive metric entry, react-native-chart-kit for bezier-smoothed health trend visualization, and react-native-vector-icons for consistent visual cues.

Localization and local storage were realized through React's Context API, allowing instant bilingual switching between Vietnamese and English across all 20 screens without requiring an app restart. React-native-async-storage was integrated to maintain persistent user authentication states, user profiles, and active language preferences locally, reducing redundant network requests and improving overall responsiveness.

Overall, the HeartHealth project demonstrates a practical application of modern software engineering principles, client-server RESTful architecture, and medical triage classification algorithms. The resulting platform serves as a reliable digital companion for daily cardiovascular monitoring and a solid baseline for future healthcare expansions.

5.2. Future Works

To further enhance the HeartHealth platform and expand its clinical utility, the following improvements are proposed for future development:

- Wearable Medical Device Connectivity: Implement Bluetooth Low Energy (BLE) protocols to interface directly with smart wearable devices and digital medical hardware (e.g., smart blood pressure cuffs, pulse oximeters, continuous glucose monitors) for automatic, friction-free vital sign synchronization.
- Caregiver & Family Portal: Develop a dedicated caregiver dashboard allowing family members or attending physicians to remotely monitor a patient's daily vitals, screening scores, and medication compliance in real-time.
- Exportable Medical PDF Reports: Build an automated PDF generation module enabling patients to export formatted weekly/monthly health summary logs and trend charts to share with cardiologists during clinic visits.
- Offline Storage & Background Sync: Integrate a local SQLite database engine to allow offline metric recording in areas without internet connectivity, automatically syncing data back to the Spring Boot server once network access is restored.

REFERENCES

[1] Oracle Corporation. (2021). Java SE 17 Archive Downloads & Documentation. https://docs.oracle.com/en/java/javase/17/

[2] VMware, Inc. (2024). Spring Boot Reference Documentation (v4.1.0). https://docs.spring.io/spring-boot/docs/current/reference/html/

[3] Meta Platforms, Inc. (2024). React Native · A framework for building native apps using React. https://reactnative.dev/docs/getting-started

[4] Oracle Corporation. (2024). MySQL 8.0 Reference Manual. https://dev.mysql.com/doc/refman/8.0/en/

[5] Red Hat, Inc. (2024). Hibernate ORM Documentation. https://hibernate.org/orm/documentation/

[6] Project Lombok. (2024). Lombok Features & Annotations Reference. https://projectlombok.org/features/

[7] React Navigation. (2024). React Navigation — Routing and navigation for React Native apps. https://reactnavigation.org/docs/getting-started

[8] Axios. (2024). Axios HTTP Client Documentation. https://axios-http.com/docs/intro

[9] React Native Chart Kit. (2023). React Native Chart Kit Documentation & Examples. https://github.com/indiespirit/react-native-chart-kit

[10] World Health Organization (WHO). (2021). Cardiovascular Diseases (CVDs) Fact Sheet & Clinical Guidelines. https://www.who.int/news-room/fact-sheets/detail/cardiovascular-diseases-(cvds)

[11] American Heart Association (AHA). (2023). Understanding Blood Pressure Readings & Risk Categories. https://www.heart.org/en/health-topics/high-blood-pressure/understanding-blood-pressure-readings